

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

ETTIway - Sistem de Navigare Interactivă Campus

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Electronică, Telecomunicații și Tehnologia Informației*
programul de studii *Rețele și Software de Telecomunicații***

Conducător științific
ș.l. dr. ing. Laurențiu BOICESCU

Absolvent
Fraticiu Vlad-Alexandru

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **FRĂTICIU O. Vlad-Alexandru, 444D**

1. Titlul temei: ETTIway. Platformă Web pentru Localizarea și Ghidarea Utilizatorilor în Campusul Universitar

2. Descrierea temei:

Proiectul va avea ca obiectiv dezvoltarea unei platforme digitale care va permite utilizatorilor să se orienteze cu ușurință în interiorul campusului universitar. Aplicația va oferi:

- Harta interactivă a campusului, cu posibilitatea de a explora clădirile din campus, sălile de laborator și cele de curs
- Facilități de editare a hărții interactive, pentru administratorii platformei
- Facilități de localizare a utilizatorului, determinată prin sistemul de poziționare al dispozitivului.
- Căutarea rapidă a sălilor, clădirilor și laboratoarelor, cu afișarea informațiilor aferente.
- Rutare și ghidare către o destinație, prin calcularea traseului și afișarea acestuia în timp real.

Proiectul va contribui la îmbunătățirea modului în care studenții, profesorii și vizitatorii vor interacționa cu spațiul universitar, oferind o soluție modernă, accesibilă și scalabilă pentru navigarea în campus.

3. Contribuția originală:

A. Modelarea campusului

- Identificarea și structurarea zonelor principale: clădiri, intrări, săli, coridoare, căi pietonale, intersecții.
- Construirea unei reprezentări a campusului sub forma unei rețele logice (graf) pentru rutare și analiză spațială.
- Definirea metadatelor asociate fiecărei locații (denumire, descriere, poziție, categorie).

B. Sistemul de localizare și afișare în timp real • Implementarea funcționalității de obținere și actualizare continuă a poziției utilizatorului.

- Afișarea poziției pe hartă și sincronizarea mișcării utilizatorului cu interfața.
- Gestionarea preciziei și a semnalelor de localizare, inclusiv situațiile de erori sau lipsă de semnal.

C. Modulul de căutare și afișare a informațiilor

- Crearea sistemului de căutare inteligentă a sălilor și clădirilor campusului.
- Afișarea detaliilor pentru destinațiile selectate și generarea opțiunilor de navigare.
- Realizarea unui mecanism de filtrare (clădire, etaj, funcție, tip sală).

D. Rutare și calculul traseului optim

- Realizarea unui algoritm de determinare a traseului optim între două puncte din campus, utilizând structura grafică internă.
- Afișarea vizuală a traseului pe hartă și actualizarea sa în timp real pe măsură ce utilizatorul se deplasează.
- Tratarea situațiilor speciale precum zone inaccesibile, lucrări, rute alternative.

E. Interfață grafică și servicii suport

- Proiectarea unui design intuitiv, adaptat pentru telefoane mobile.
- Proiectarea și implementarea bazei de date aferente
- Integrarea panourilor de informații, meniurilor, funcțiilor de control și feedback vizual.
- Optimizarea timpilor de încărcare și a performanței generale a aplicației.

4. Resurse și bibliografie:

[Madkour et al., 2017] – A Survey of Shortest-Path Algorithms. ResearchGate, 2017.
(https://www.researchgate.net/publication/316736456_A_Survey_of_ShortestPath_Algorithms DOI:
10.48550/arXiv.1705.02044)

[Chen, 2022] – Dijkstra's method with min-priority queue optimization. arXiv, 2022.
(<https://arxiv.org/pdf/2208.00312.pdf>)

[Luxen, 2011] – Real-time routing with OpenStreetMap data (DOI: 10.1145/2093973.2094062)

5. Data înregistrării temei: 2026-01-04 21:28:42

Conducător(i) lucrare,

Ș. L. Dr. Ing. Laurențiu BOICESCU

Student,

FRĂTICIU O. Vlad-Alexandru

Director departament,

Conf. dr. ing. Șerban OBREJA

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **97d3a2f81f**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *ETTIway - Sistem de Navigare Interactivă Campus*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Rețele și Software de Telecomunicații* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Fraticiu Vlad-Alexandru

.....

Cuprins

Lista acronimelor	viii
Introducere	1
0.1. Obiectivele generale ale proiectului	1
0.2. Metodologia folosită	1
0.3. Contribuția studentului și rezultatele obținute	2
1. Fundamente teoretice și tehnologii utilizate	4
1.1. Sisteme de navigare și hărți digitale	4
1.1.1. Reprezentarea hărților prin dale (tiles)	4
1.2. Navigarea exterioară și navigarea interioară	5
1.2.1. Navigarea exterioară	5
1.2.2. Navigarea interioară	5
1.2.3. Integrarea celor două niveluri	6
1.3. Date geo-spațiale și modelarea informației geografice	6
1.3.1. Sisteme de coordonate și sisteme de referință spațială	6
1.3.2. Reprezentarea geometriilor: WKT și WKB	7
1.3.3. OpenStreetMap ca sursă de date	7
1.4. Poziționarea și localizarea utilizatorului	7
1.4.1. Sistemul de poziționare globală	8
1.4.2. Precizia și limitările localizării	8
1.5. Algoritmi de determinare a drumului minim	9
1.5.1. Modelarea problemei prin grafuri	9
1.5.2. Algoritmul Dijkstra	9
1.5.3. Performanță și optimizări	10
1.6. Interfețe pentru dispozitive mobile	10
1.7. Arhitecturi de aplicații web	12
1.7.1. Separarea frontend-backend	12
1.7.2. Servicii REST și formatul JSON	12
1.7.3. Rețele private virtuale și topologii de tip mesh	12
1.8. Tehnologii utilizate în implementare	13
1.8.1. Spring Boot pentru backend	13
1.8.2. Leaflet și JavaScript pentru frontend	14
1.8.3. Organizarea datelor și extensibilitatea	14
1.9. Concluziile capitolului	14
2. Arhitectura sistemului	15
2.1. Vedere de ansamblu asupra arhitecturii	15
2.2. Nivelul de servicii (backend)	16
2.2.1. Punctul de intrare al aplicației	16
2.2.2. Accesul la date prin JPA	17

2.2.3.	Strategia de tratare a erorilor	17
2.3.	Nivelul de date	17
2.3.1.	Alegerea bazei de date	18
2.3.2.	Entitatea utilizator	18
2.3.3.	Entitatea pentru datele hărții	19
2.3.4.	Reprezentarea planurilor de etaj	19
2.3.5.	Strategia de modelare a datelor	19
2.4.	Securitate: autentificare și autorizare	20
2.4.1.	Criptarea parolelor	20
2.4.2.	Configurarea regulilor de acces	21
2.4.3.	Încărcarea utilizatorilor și inițializarea contului de administrator	21
2.5.	Nivelul de prezentare (frontend)	21
2.5.1.	Inițializarea hărții și delimitarea zonei	22
2.5.2.	Construirea grafului de navigare	22
2.5.3.	Calculul traseului prin algoritmul Dijkstra	22
2.6.	Fluxul de comunicare între componente	23
2.7.	Concluziile capitolului	24
3.	Implementarea funcționalităților și punerea în funcțiune	25
3.1.	Gestionarea rolurilor și modurile de utilizare	25
3.2.	Editarea hărții de către administrator	26
3.3.	Căutarea și afișarea informațiilor	27
3.4.	Securitatea la nivelul interfeței	27
3.5.	Rutarea exterioară și afișarea vizuală a traseului	28
3.6.	Rutarea în interiorul clădirilor	28
3.6.1.	Modelul de date al planurilor de etaj	29
3.6.2.	Editarea planurilor și salvarea în baza de date	29
3.6.3.	Calculul traseului indoor	29
3.6.4.	Detectarea dispozitivului și adaptarea interfeței	30
3.6.5.	Gestionarea stării partajate în aplicație	30
3.6.6.	Verificarea autorizării pe două niveluri	30
3.7.	Punerea în funcțiune prin rețea privată virtuală	31
3.8.	Configurarea serverului	31
3.8.1.	Rețeaua privată cu Tailscale	32
3.8.2.	Adresarea prin MagicDNS	33
3.8.3.	Criptarea HTTPS și accesul la GPS	33
3.9.	Gestionarea serviciilor pe server	33
3.10.	Concluziile capitolului	34
4.	Testarea și evaluarea sistemului	35
4.1.	Metodologia de testare	35
4.2.	Scenarii de testare funcțională	36
4.2.1.	Autentificarea și diferențierea rolurilor	36
4.2.2.	Căutarea și afișarea detaliilor	36
4.2.3.	Editarea și persistarea modificărilor	36
4.2.4.	Calculul și afișarea traseului	37
4.2.5.	Editarea și rutarea în interior	37

4.3. Testarea pe mai multe dispozitive și browsere	37
4.4. Comportamentul în condiții reale de utilizare	38
4.5. Limitări identificate	38
4.5.1. Precizia poziționării în interiorul clădirilor	38
4.5.2. Acoperirea actuală a datelor	39
4.6. Concluziile capitolului	39
Concluzii	40
Sinteza rezultatelor	40
Opinia personală asupra rezultatelor	41
Direcții viitoare de dezvoltare	41
Reflecție finală	42
Bibliografie	43
Anexa A. Cod Java (Backend)	45
Anexa B. Cod JavaScript (Frontend)	50

Lista acronimelor

API	Application Programming Interface
BCrypt	Blowfish Cipher (algoritm de criptare a parolelor)
CSS	Cascading Style Sheets
CSRF	Cross-Site Request Forgery
DOI	Digital Object Identifier
DNS	Domain Name System
ETTI	Electronică, Telecomunicații și Tehnologia Informației
GeoJSON	Geographic JavaScript Object Notation
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
OSM	OpenStreetMap
REST	Representational State Transfer
RST	Rețele și Software de Telecomunicații
SRS	Spatial Reference System
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
VPN	Virtual Private Network
Wi-Fi	Wireless Fidelity
WKB	Well-Known Binary
WKT	Well-Known Text
XSS	Cross-Site Scripting

Introducere

Campusurile universitare moderne reprezintă spații complexe, formate din zeci de clădiri, sute de săli și o rețea densă de alei și puncte de interes. Orientarea într-un astfel de mediu, în special pentru studenții din primii ani sau pentru vizitatori, este adesea dificilă. Identificarea unei clădiri după denumirea uzuală, găsirea celui mai scurt drum pe alei sau localizarea unei anumite săli într-un corp pe care nu îl cunoști sunt sarcini care pot consuma timp și pot duce la întârzieri sau confuzii. Soluțiile clasice de hărți (Google Maps, OpenStreetMap) acoperă bine deplasarea între adrese, dar nu sunt adaptate la nivelul de detaliu necesar într-un campus. Acestea nu arată sălile, laboratoarele și caracteristicile lor, iar planificarea unui drum pe aleile interne nu este întotdeauna corectă.

Plecând de la această observație, lucrarea propune ETTIway, o platformă web dezvoltată în acest sens, pentru navigarea într-un campus universitar. Aplicația oferă atât o componentă pentru spațiul exterior, cu hartă, căutare și calculul traseelor de pe aleile campusului, cât și o componentă pentru spațiul interior al clădirilor, cu planuri de etaj care se pot edita și rutare între camere pe același etaj.

0.1 Obiectivele generale ale proiectului

Obiectivul principal al proiectului a fost dezvoltarea unei platforme web funcționale, ușor de utilizat de pe dispozitive mobile, care să acopere nevoile specifice de orientare într-un campus universitar. Pentru atingerea acestui scop au fost formulate mai multe obiective secundare, care au ghidat deciziile de proiectare pe tot parcursul dezvoltării.

Primul obiectiv secundar a fost realizarea unei interfețe de hartă interactivă pentru spațiul exterior al campusului, peste un set de date provenit din OpenStreetMap, cu posibilitatea de a calcula traseul optim între două puncte prin algoritmul lui Dijkstra. Al doilea obiectiv a fost implementarea unei componente dedicate pentru spațiul interior, în care un administrator poate desena planurile de etaj ale clădirilor. Urmând ca un utilizator obișnuit să poată căuta o cameră și să poată obține traseul către aceasta pe planul respectiv. Al treilea obiectiv a fost separarea clară a rolurilor de utilizator (administrator și utilizator obișnuit) printr-un mecanism de autentificare solid, care să permită editarea hărții doar de către persoane autorizate. Al patrulea obiectiv a fost asigurarea unui mecanism de acces sigur la aplicație, fără expunerea serverului pe internetul public, prin folosirea unei rețele private de tip mesh.

Pe lângă obiectivele funcționale, au fost urmărite și obiective tehnice: codul să fie organizat în niveluri clar separate (prezentare, servicii, date). Datele să poată fi extinse fără modificări de schemă în baza de date, iar aplicația să funcționeze corect pe o varietate de dispozitive, de la laptopuri și până la telefoane mobile cu acces la internet.

0.2 Metodologia folosită

Dezvoltarea platformei ETTIway a urmat o metodologie incrementală, în care fiecare componentă a fost proiectată, implementată și testată separat, înainte de a fi integrată în fluxul de ansamblu al aplicației. Această abordare a permis identificarea și corectarea din timp a problemelor și a asigurat o evoluție controlată a proiectului.

În prima etapă au fost studiate fundamentele teoretice și tehnologiile disponibile, cu accent pe sistemele de coordonate geografice, formatele de date geo-spațiale (în special GeoJSON), algoritmi de calcul al drumului minim în grafuri și framework-urile web moderne pentru construirea aplicațiilor de tip client-server. În urma acestei documentări am ales tehnologiile folosite: Spring Boot pentru backend, datorită maturității ecosistemului Java și a suportului foarte bun pentru securitate; PostgreSQL pentru stocare, pentru fiabilitatea sa și pentru suportul nativ al câmpurilor text de dimensiune mare. Leaflet pentru afișarea hărții, datorită faptului că este o unealtă open-source și pentru suportul nativ al gesturilor tactile pe dispozitivele mobile.

În a doua etapă am construit componenta exterioară: încărcarea unei hărți OpenStreetMap centrate pe campusul universitar, definirea rețelei de alei sub forma unui graf editabil de către administrator, și implementarea algoritmului lui Dijkstra în JavaScript pentru calculul traseelor între două puncte. Persistența grafului a fost realizată într-o coloană text de tip JSON a unei entități unice MapData, decizie de proiectare orientată spre flexibilitate.

În a treia etapă am extins aplicația cu o componentă pentru spațiul interior al clădirilor. Pentru această componentă am proiectat un model de date separat, structurat ierarhic pe clădiri, etaje și elemente (camere, noduri, coridoare), păstrat în aceeași entitate MapData prin adăugarea unei a doua coloane JSON. Rutarea în interior folosește o instanță dedicată a algoritmului Dijkstra, adaptată pentru lucrul cu coordonate sub formă de pixeli în loc de coordonate geografice.

În a patra etapă am abordat aspectele legate de deployment și securitate. Am configurat un server propriu pentru găzduirea aplicației și am ales folosirea rețelei private virtuale Tailscale pentru a evita expunerea serverului pe internetul public. Această decizie a fost luată în mod intenționat, pentru un proiect academic, cu un număr limitat de utilizatori, o rețea privată oferă atât simplitate de configurare cât și un nivel ridicat de securitate, fără necesitatea unor măsuri suplimentare specifice expunerii publice. Certificatele HTTPS pe domeniul `.ts.net` sunt obținute automat, fără configurări manuale.

Pe tot parcursul dezvoltării am acordat o atenție constantă testării. Fiecare componentă nouă a fost verificată separat, prin scenarii reprezentative, iar la finalul fiecărei etape a fost realizată o testare de integrare, inclusiv prin folosirea efectivă a aplicației în campus, pe un dispozitiv mobil conectat la rețeaua privată.

0.3 Contribuția studentului și rezultatele obținute

Lucrarea de față prezintă o aplicație web pe care am dezvoltat-o integral, de la analiza cerințelor până la implementare, testare și deployment. Contribuția mea acoperă toate componentele platformei.

La nivelul de backend, am implementat structura proiectului Spring Boot, entitățile JPA, repository-urile pentru accesul la date, configurația de securitate cu Spring Security și rolurile de administrator și utilizator obișnuit, precum și controlerele REST care expun serviciile aplicației.

La nivelul de frontend, am scris codul JavaScript pentru afișarea hărții interactive, am implementat algoritmul lui Dijkstra atât pentru rutarea exterioară, cât și pentru rutarea în interior. Am construit modulele de editare a hărții cu ajutorul bibliotecii Leaflet, am proiectat interfața de căutare cu sugestii dinamice și am gestionat detectarea dispozitivului mobil pentru o experiență adaptată.

La nivelul de date, am structurat modelul folosit pentru graf și pentru planurile indoor, am ales formatul GeoJSON ca standard intern de schimb, și am implementat fluxurile de salvare și încărcare a datelor între client și server.

La nivelul de deployment, am configurat serverul propriu, am instalat și configurat rețeaua privată Tailscale și am realizat conectarea dispozitivelor mobile pentru testare în condiții reale.

Rezultatul este o aplicație funcțională, care a fost testată în campus și care răspunde tuturor obiectivelor formulate la începutul proiectului. Componenta exterioară permite căutarea clădirilor, calculul traseelor pe alei și afișarea poziției utilizatorului pe hartă. Componenta interioară permite editarea planurilor de etaj de către administrator și calculul traseelor între camere pentru utilizatorul obișnuit. Mecanismul de autentificare separă corect rolurile, iar accesul prin rețea privată asigură atât securitatea, cât și posibilitatea utilizării aplicației de pe dispozitive mobile.

Lucrarea este structurată în patru capitole, urmate de concluzii. Capitolul 1 prezintă fundamentele teoretice și tehnologiile pe care se bazează ETTIway. Capitolul 2 descrie arhitectura sistemului, cu cele trei niveluri și deciziile de proiectare adoptate. Capitolul 3 detaliază implementarea funcționalităților, atât pentru spațiul exterior, cât și pentru spațiul interior. Capitolul 4 prezintă metodologia și rezultatele testării. Concluziile sintetizează rezultatele obținute și schițează direcțiile de dezvoltare ulterioară. Codul sursă este disponibil în anexele A (componente Java) și B (componente JavaScript).

Capitolul 1

Fundamente teoretice și tehnologii utilizate

Acest capitol prezintă fundamentul teoretic și tehnologic pe care se sprijină platforma ETTIway. Sunt abordate, în mod gradual, conceptele de bază ale sistemelor de navigare digitală, particularitățile navigării în spații exterioare și interioare, modul de reprezentare a datelor geo-spațiale, precum și tehnologiile concrete folosite în implementare. Scopul acestui capitol este de a oferi contextul necesar pentru a înțelege deciziile de proiectare descrise în capitolele următoare. Vom pleca de la principiile generale și vom ajunge la instrumentele specifice care au stat la baza dezvoltării aplicației.

1.1 Sisteme de navigare și hărți digitale

Un sistem de navigare digitală reprezintă un ansamblu de componente software și date care permit unui utilizator să se localizeze într-un spațiu, să identifice puncte de interes și să determine trasee între acestea. Spre deosebire de hărțile tipărite, hărțile digitale sunt interactive, astfel încât utilizatorul poate mări sau micșora nivelul de detaliu, poate filtra informația afișată și poate interoga elementele de pe hartă pentru a obține date suplimentare. Acest lucru este esențial într-un mediu complex, cum este un campus universitar. În locațiile unde densitatea informației relevante este ridicată, iar nevoile utilizatorilor variază considerabil de la o situație la alta.

În general, un sistem de navigare modern se construiește pe trei piloni fundamentali: un strat de date geografice care descrie spațiul fizic, un motor de procesare care interpretează aceste date și o interfață de vizualizare care prezintă informația într-o formă accesibilă utilizatorului. Calitatea experienței finale depinde de modul în care aceste trei niveluri sunt corelate și optimizate. Un strat de date incomplet conduce la rezultate inexacte, un motor de procesare lent degradează experiența interactivă, iar o interfață confuză anulează beneficiile unei baze de date bine structurate.

Hărțile digitale interactive au cunoscut o dezvoltare accentuată odată cu apariția bibliotecilor de cartografie web, care permit afișarea hărților direct în browser, fără instalarea unor aplicații dedicate. Acest model, în care harta este prezentată ca o aplicație web are avantaje semnificative din punct de vedere al accesibilității. Utilizatorul are nevoie doar de un browser modern și de o conexiune la internet, fără să depindă de un sistem de operare sau de hardware specializat. ETTIway adoptă tocmai acest model, fiind o aplicație web ce poate fi accesată de pe orice dispozitiv cu un browser compatibil.

1.1.1 Reprezentarea hărților prin dale (tiles)

Una dintre tehnicile fundamentale folosite în cartografia web este împărțirea hărții în dale (în engleză *tiles*), imagini pătrate de dimensiune fixă, de regulă 256 pe 256 de pixeli, care acoperă o anumită zonă geografică la un anumit nivel de zoom. În loc să se încarce o singură imagine de dimensiune foarte mare, harta este compusă dinamic din mai multe asemenea dale, dintre care se afișează doar cele vizibile în fereastra curentă. Pe măsură ce utilizatorul se deplasează

sau modifică nivelul de zoom, sunt încărcate noile dale necesare, iar cele care ies din câmpul vizual sunt eliberate din memorie.

Acest mecanism de afișare are mai multe avantaje practice. În primul rând, reduce volumul de date transferat la un moment dat, deoarece se transmit doar porțiunile vizibile ale hărții. Permite stocarea în memoria cache a dalelor deja descărcate, ceea ce accelerează navigarea repetată prin aceeași zonă. De asemenea, sistemul de dale se pretează foarte bine la mai multe niveluri de detaliu cum ar fi, la niveluri mici de zoom se afișează contururi generale, iar la niveluri mari apar detalii fine, precum clădiri individuale, intrări sau trasee pietonale. Această organizare a datelor este folosită de majoritatea serviciilor de hărți web și constituie fundamentul pe care se construiește și experiența vizuală din ETTIway.

1.2 Navigarea exterioară și navigarea interioară

O distincție esențială în domeniul navigării este cea dintre navigarea exterioară și cea interioară. Cele două tipuri de navigare ridică probleme diferite atât din punct de vedere al modalității de procurare a datelor, cât și din punct de vedere al modului de interacțiune cu utilizatorul. Una dintre contribuțiile centrale ale lucrării de față este tocmai integrarea acestor două dimensiuni, astfel încât utilizatorul să poată trece în mod natural de la nivelul campusului la nivelul unei clădiri și, mai departe, la nivelul unei săli sau al unui laborator.

1.2.1 Navigarea exterioară

Navigarea exterioară se referă la orientarea în spațiul deschis: identificarea clădirilor, a aleilor, a intrărilor și a traseelor dintre diferite puncte de interes ale campusului. La acest nivel, datele geografice sunt în general disponibile prin servicii cartografice publice și pot fi reprezentate în sisteme de coordonate geografice standard, exprimate prin latitudine și longitudine. Pozițiile sunt asociate unui sistem global de referință, ceea ce permite suprapunerea corectă a diferitelor straturi de informație și compatibilitatea cu datele provenite din surse externe.

Pentru un campus universitar, navigarea exterioară presupune reprezentarea fidelă a infrastructurii: conturul clădirilor, denumirile acestora, aleile pietonale, drumurile de acces și punctele de interes precum intrările principale, bibliotecile, cantinele sau punctele de orientare. Provocarea principală la acest nivel nu este atât tehnologică, cât legată de acuratețea datelor: o hartă exterioară este utilă doar în măsura în care reflectă corect realitatea din teren și este actualizată atunci când infrastructura se modifică.

1.2.2 Navigarea interioară

Navigarea interioară se ocupă de orientarea în interiorul clădirilor: identificarea sălilor, a laboratoarelor, a etajelor și a traseelor dintre acestea. Acest tip de navigare ridică dificultăți suplimentare în comparație cu cea exterioară. Pe de o parte, datele de interior nu sunt, în general, disponibile în servicii cartografice publice și trebuie construite manual, plecând de la planuri ale clădirilor. Pe de altă parte, reprezentarea spațiului interior implică o componentă verticală, etajele, care nu poate fi exprimată printr-o simplă pereche de coordonate plane, ci necesită un mecanism de organizare pe niveluri.

În cadrul ETTIway, navigarea interioară este tratată prin asocierea fiecărei clădiri cu un set de planuri de etaj, fiecare plan corespunzând unui nivel distinct al clădirii. Utilizatorul poate selecta o clădire de pe harta exterioară și poate apoi naviga pe etajele acesteia, vizualizând dispunerea sălilor și a spațiilor relevante. Această abordare, în care nivelul exterior și cel interior sunt corelate prin intermediul entității „clădire”, permite o tranziție coerentă între cele două perspective și constituie nucleul funcțional al platformei.

1.2.3 Integrarea celor două niveluri

Marea provocare a unui sistem complet de navigare în campus este unificarea celor două niveluri într-o singură experiență, fără discontinuități. În multe soluții existente, navigarea exterioară și cea interioară sunt tratate de aplicații separate sau de surse de date necorelate, ceea ce obligă utilizatorul să combine manual informații provenite din mai multe locuri. ETTIway propune o abordare integrată, în care o clădire selectată pe harta exterioară devine punctul de intrare către reprezentarea interioară a acesteia, păstrând astfel un fir narativ continuu pentru utilizator. Această integrare reprezintă una dintre contribuțiile practice principale ale lucrării și este detaliată în capitolele dedicate arhitecturii și implementării.

1.3 Date geo-spațiale și modelarea informației geografice

Fundamentul oricărui sistem de navigare îl constituie datele geo-spațiale, adică informația care descrie poziția și forma obiectelor în spațiu. Modul în care aceste date sunt structurate, stocate și interogate influențează direct performanța și flexibilitatea întregului sistem. În această secțiune sunt prezentate conceptele de bază necesare înțelegerii stratului de date al aplicației ETTIway.

1.3.1 Sisteme de coordonate și sisteme de referință spațială

Pentru a putea localiza un obiect pe suprafața Pământului este necesar un sistem de referință spațială (SRS). Acesta definește modul în care coordonatele numerice se traduc în poziții reale. Cel mai răspândit sistem de referință folosit în aplicațiile web este cel bazat pe latitudine și longitudine, exprimate în grade, asociat unui model standardizat al Pământului. Acest sistem este utilizat pe scară largă deoarece este compatibil cu majoritatea surselor de date publice și cu serviciile de poziționare prin satelit.

Pe lângă sistemul geografic exprimat în grade, în cartografia web este frecvent folosită și o proiecție plană, care transformă suprafața curbă a Pământului într-un plan, facilitând afișarea hărților pe ecran. Trecerea între diferite sisteme de referință se numește reproiecție și trebuie tratată cu atenție, deoarece o suprapunere incorectă a straturilor de date conduce la afișarea eronată a obiectelor. În cadrul ETTIway, consecvența sistemului de referință între datele provenite din diverse surse este o condiție necesară pentru alinierea corectă a infrastructurii campusului.

Un aspect tehnic important, ușor de trecut cu vederea dar cu consecințe practice serioase, este **ordinea coordonatelor** folosită de diferite biblioteci și formate. În standardul GeoJSON, folosit pentru schimbul de date geografice între componente, perechea de coordonate este scrisă în ordinea `[longitudine, latitudine]`, conform convenției matematice `[x, y]`. Pe de altă parte, biblioteca Leaflet, folosită pentru afișarea hărții în ETTIway, folosește ordinea `[latitudine, longitudine]`, conform convenției geografice clasice. Această asimetrie reprezintă o sursă frecventă de erori în aplicațiile care procesează date geo-spațiale, iar identificarea corectă a ordinii așteptate de fiecare strat este esențială pentru afișarea corectă a geometriilor. În implementarea ETTIway, conversia între cele două convenții se face explicit la nivelul fiecărei interacțiuni cu date GeoJSON, pentru a evita inversiunea accidentală care ar plasa, de exemplu, un punct din București în mijlocul oceanului.

ETTIway folosește sistemul de coordonate WGS84 (World Geodetic System 1984), standardul global pentru localizarea punctelor pe Pământ. Acest sistem este folosit de GPS, Google Maps, OpenStreetMap și majoritatea aplicațiilor cu hărți.

Pe lângă WGS84, există sute de sisteme de coordonate proiectate locale, optimizate pentru regiuni geografice specifice, care minimizează distorsiunile de distanță sau de suprafață într-o

anumită zonă. Pentru România, sistemul oficial folosit în cartografie și topografie este **Stereo70** (EPSG:3844), o proiecție conică secantă optimizată pentru teritoriul național. Acest sistem nu este folosit direct în ETTIway, deoarece datele de pornire provin din OpenStreetMap (care folosește WGS84) și serviciul de localizare al browserului raportează tot în WGS84, însă cunoașterea sa este utilă într-un context mai larg, în care aplicația ar fi integrată cu sisteme administrative locale care folosesc Stereo70 (cum sunt platformele cadastrale).

1.3.2 Reprezentarea geometriilor: WKT și WKB

Obiectele geografice, puncte, linii și poligoane, sunt descrise prin geometrii. O clădire poate fi reprezentată ca un poligon, o alee ca o linie, iar un punct de interes ca un singur punct. Pentru a stoca și a transmite aceste geometrii există formate standardizate. Formatul textual WKT (*Well-Known Text*) descrie geometriile într-o formă lizibilă pentru om. În timp ce formatul binar WKB (*Well-Known Binary*) le codifică într-o reprezentare compactă, optimizată pentru stocare și transfer. Ambele formate sunt larg acceptate de bazele de date spațiale și de bibliotecile de procesare geografică, ceea ce asigură compatibilitatea între componentele sistemului.

Folosirea unor formate standardizate de reprezentare a geometriilor este importantă pentru a putea extinde aplicația: datele pot fi importate din surse externe, prelucrate cu instrumente specializate și apoi consumate de aplicație. Acest aspect susține unul dintre obiectivele proiectului, și anume capacitatea de a adăuga ușor noi clădiri și zone fără modificări majore în logica aplicației.

1.3.3 OpenStreetMap ca sursă de date

OpenStreetMap (OSM) este o baza de date care pune la dispoziție date geografice open-source [6], contribuie și întreținute de o comunitate largă de utilizatori din întreaga lume. Datele OSM acoperă o gamă variată de elemente: drumuri, clădiri, alei pietonale, puncte de interes și multe altele. Aceste date sunt deschise (open-source). Din aceasta cauză pot fi descărcate și prelucrate liber, ceea ce le face o sursă valoroasă pentru aplicațiile de navigare. În special în contextul în care construirea de la zero a unui set complet de date geografice ar fi extrem de costisitoare.

În cadrul ETTIway, datele provenite din OpenStreetMap sunt folosite pentru reprezentarea infrastructurii exterioare a campusului, în special pentru afișarea drumurilor și a aleilor. Modelul de date OSM organizează informația în jurul a trei tipuri de elemente fundamentale. Primul ar fi nodurile, care reprezintă puncte individuale definite prin coordonate. Al doilea ar fi căile, care reprezintă secvențe ordonate de noduri și pot descrie atât linii, cât și conturul unor suprafețe. Al treilea tip sunt relațiile, care grupează mai multe elemente pentru a descrie structuri complexe. Fiecărui element *i* se pot asocia etichete sub formă de perechi cheie-valoare, care îi descriu semnificația. De exemplu, tipul de drum sau funcția unei clădiri. Această structură flexibilă permite filtrarea și selectarea elementelor relevante pentru navigarea în campus. Datele extrase din OpenStreetMap sunt adaptate scenariilor de navigare din campus, fiind reținute doar elementele relevante, drumuri, alei și contururi de clădiri. Informațiile care nu prezintă interes pentru aplicație sunt eliminate. Organizarea atentă a stratului de date reprezintă, de altfel, o parte semnificativă a contribuției proprii descrise în lucrare.

1.4 Poziționarea și localizarea utilizatorului

Pe lângă reprezentarea statică a infrastructurii campusului, un sistem de navigare complet trebuie să poată determina poziția curentă a utilizatorului și să o raporteze la harta afișată.

Această capacitate transformă harta dintr-un simplu instrument de consultare într-un asistent de navigare personalizat, capabil să indice nu doar unde se află o destinație, ci și cum poate fi atinsă pornind din locația curentă.

1.4.1 Sistemul de poziționare globală

Determinarea poziției în spațiul exterior se bazează, de obicei, pe sistemul de poziționare globală (GPS). Aceasta utilizează o infrastructură de sateliți care permite unui receptor să își calculeze coordonatele pe baza semnalelor recepționate. Fiecare satelit transmite continuu informații privind poziția sa și momentul exact al emisiei. Receptorul măsoară diferențele de timp dintre semnalele provenite de la mai mulți sateliți. Cu aceste diferențe de timp, cu ajutorul triangulației se deduce propria poziție în coordonate de latitudine și longitudine. Pentru o estimare bidimensională a poziției sunt necesare semnale de la cel puțin trei sateliți, iar pentru includerea altitudinii este necesar un al patrulea.

În aplicațiile web, accesul la poziția dispozitivului nu se realizează direct cu hardware-ul GPS, ci printr-o interfață standardizată pusă la dispoziție de browser, care abstractizează sursa concretă a informației de localizare. Browserul poate combina mai multe surse, semnalul GPS propriu-zis, rețelele de telefonie mobilă și punctele de acces de tip Wi-Fi, pentru a oferi o estimare a poziției chiar și atunci când semnalul satelitar este slab. Această interfață furnizează, pe lângă coordonatele propriu-zise, și o estimare a preciziei, exprimată ca o rază de incertitudine în jurul poziției raportate.

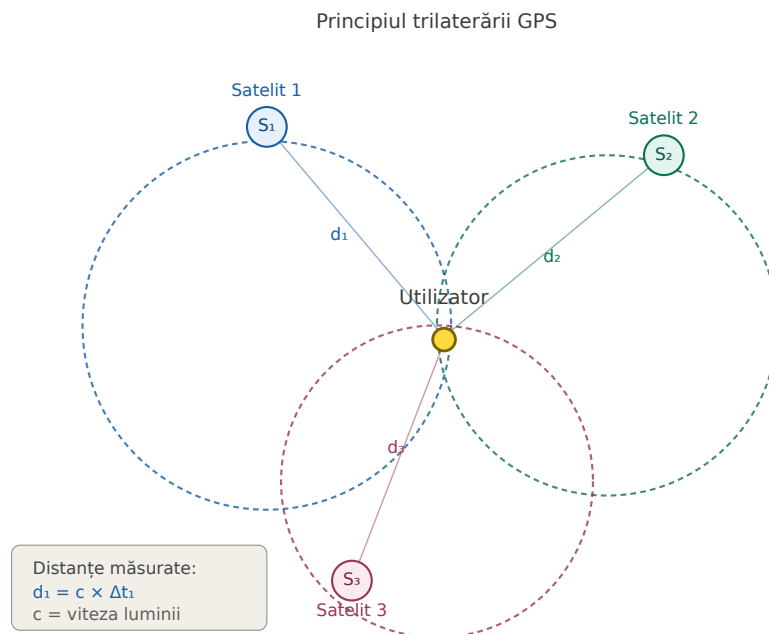


Figura 1.1: Principiul trilaterării GPS

Este important de menționat că platforma ETTIway nu implementează principiul de trilaterare. Procesul matematic de calcul al poziției pe baza semnalelor recepționate de la sateliți este realizat la un nivel mult mai jos, în chipset-ul GPS al dispozitivului mobil.

1.4.2 Precizia și limitările localizării

Precizia localizării variază semnificativ în funcție de mediu și de sursele disponibile. În spațiu deschis, cu vizibilitate directă către sateliți, eroarea este de ordinul câtorva metri. În schimb,

în interiorul clădirilor sau în apropierea structurilor înalte, semnalul provenit de la sateliți este atenuat ori reflectat. Precizia este degradată considerabil din aceste cauze, putând ajunge la zeci de metri. Acest fenomen este deosebit de relevant pentru un campus universitar, unde clădirile sunt dese și utilizatorii petrec mult timp în interior, exact contextul în care poziționarea prin semnalele provenite de la sateliți devine cel mai puțin fiabilă.

Din această cauză, un sistem de navigare solid trebuie să trateze explicit situațiile de imprecizie sau de lipsă temporară a semnalului, evitând afișarea unei poziții înșelător de exacte și informând utilizatorul cu privire la gradul de încredere al localizării. În cadrul ETTIway, poziția utilizatorului este folosită ca punct de plecare pentru calculul traseului, fiind asociată celui mai apropiat punct de pe rețeaua de trasee a campusului, astfel încât micile erori de localizare să nu împiedice funcționarea rutării.

1.5 Algoritmi de determinare a drumului minim

Componenta de rutare reprezintă nucleul funcțional al oricărui sistem de navigare. Odată ce infrastructura campusului este reprezentată sub forma unui graf, problema găsirii celui mai scurt traseu între două puncte devine o problemă clasică de teoria grafurilor: determinarea drumului de cost minim între două noduri. Această secțiune prezintă fundamentele teoretice ale problemei și algoritmi care o rezolvă. Vom pune accent pe soluția folosită în implementare.

1.5.1 Modelarea problemei prin grafuri

Un graf este o structură formată dintr-un set de noduri și un set de muchii care leagă perechi de noduri. În contextul navigării, nodurile corespund punctelor de decizie, intersecții, intrări, capete de alei, iar muchiile corespund segmentelor de traseu care le unesc. Fiecărei muchii i se asociază o pondere, care exprimă costul parcurgerii segmentului respectiv; în cazul cel mai simplu, ponderea este distanța geografică reală dintre cele două noduri, dar pot fi folosite și alte criterii, precum timpul estimat de parcurgere sau gradul de accesibilitate.

Atunci când ponderile reprezintă distanțe, graful este nedirecționat, deoarece o alee poate fi parcursă în ambele sensuri cu același cost. Problema centrală devine astfel găsirea, în acest graf ponderat, a lanțului de muchii care leagă nodul de start de nodul de destinație și a cărui sumă a ponderilor este minimă. Această formulare permite aplicarea unor algoritmi cunoscuți. [1]

1.5.2 Algoritmul Dijkstra

Cel mai cunoscut algoritm pentru determinarea drumului minim de la un nod sursă către celelalte noduri ale unui graf cu ponderi nenegative este algoritmul Dijkstra. Principiul său de funcționare este unul de tip lacom (*greedy*). Algoritmul menține, pentru fiecare nod, cea mai mică distanță cunoscută până la acel moment față de nodul sursă, precum și nodul precedent pe traseul optim. La fiecare pas, este selectat nodul nevizitat cu distanța minimă, după care se încearcă îmbunătățirea distanțelor către vecinii săi, operație cunoscută sub numele de relaxare a muchiilor. Procesul continuă până când nodul de destinație este atins sau până când toate nodurile accesibile au fost vizitate.

Corectitudinea algoritmului se bazează pe observația că, atunci când un nod este selectat ca având distanța minimă dintre cele nevizitate, această distanță este deja optimă și nu mai poate fi îmbunătățită ulterior, întrucât toate ponderile sunt nenegative. La final, traseul optim se reconstruiește pornind de la nodul de destinație și urmărind, în sens invers, lanțul nodurilor precedente până la sursă. Acest mod de funcționare îl face potrivit pentru calculul traseelor într-o rețea de dimensiuni moderate, cum este cea a unui campus universitar.

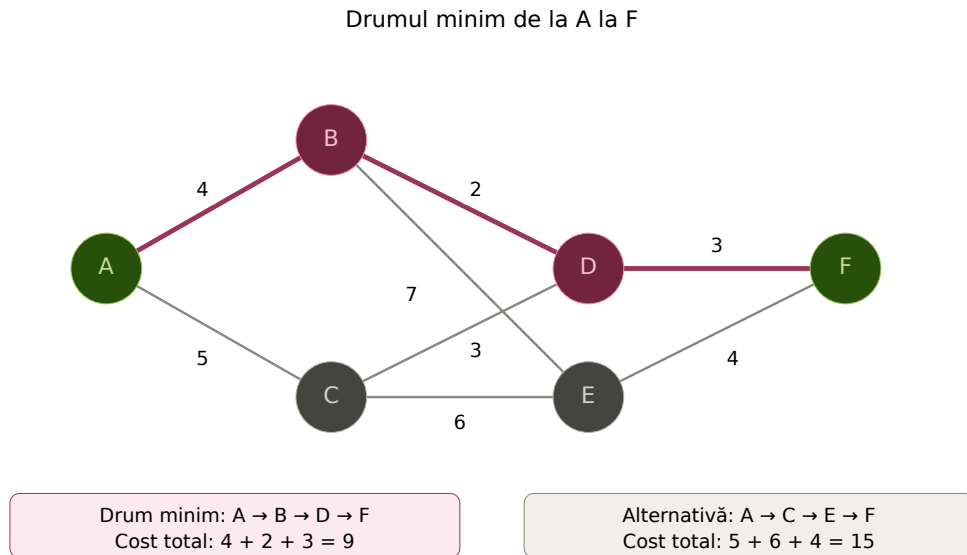


Figura 1.2: Exemplu de aplicare a algoritmului lui Dijkstra: drumul minim de la nodul A la nodul F este evidențiat cu roșu. Algoritmul găsește traseul $A \rightarrow B \rightarrow D \rightarrow F$ cu cost total 9, în detrimentul alternativei mai lungi prin C și E (cost 15).

Pentru a ilustra modul de funcționare a algoritmului, Figura 1.2 prezintă un graf cu șase noduri (A până la F) și un set de muchii cu costuri asociate. Algoritmul lui Dijkstra explorează sistematic toate căile posibile de la nodul de plecare și identifică drumul de cost minim către fiecare nod destinație. Pentru exemplul ilustrat, drumul de la A la F cu cost minim este $A \rightarrow B \rightarrow D \rightarrow F$, cu un cost total de 9 unități, în detrimentul alternativei $A \rightarrow C \rightarrow E \rightarrow F$, care are un cost total de 15 unități.

1.5.3 Performanță și optimizări

Performanța algoritmului Dijkstra depinde în mod esențial de modul în care este implementată operația de selectare a nodului cu distanța minimă. Într-o implementare directă, în care la fiecare pas se parcurg toate nodurile pentru a-l găsi pe cel cu distanța minimă, complexitatea devine pătratică în raport cu numărul de noduri. Pentru grafuri mari, această abordare devine ineficientă, iar literatura de specialitate propune optimizări bazate pe structuri de date de tip coadă de prioritate, care reduc semnificativ timpul de execuție prin extragerea rapidă a nodului minim [2].

Pentru rețeaua unui campus universitar, în care numărul de noduri este relativ redus, implementarea directă a algoritmului oferă timpi de răspuns acceptabili, calculul traseului fiind perceput ca instantaneu de către utilizator. Tocmai din acest motiv, în cadrul ETTIway s-a optat pentru o implementare clară și ușor de urmărit a algoritmului, prioritizând lizibilitatea codului în detrimentul unor optimizări care nu ar fi adus beneficii practice la această scară. Aceleași principii stau la baza serviciilor de rutare la scară largă construite peste date deschise precum cele din OpenStreetMap [3], care însă necesită structuri de date și optimizări mult mai complexe pentru a trata rețele de dimensiunea unor orașe întregi.

1.6 Interfețe pentru dispozitive mobile

Un sistem de navigare în campus este utilizat, natural, preponderent de pe dispozitive mobile, utilizatorul având nevoie de orientare tocmai atunci când se deplasează prin campus, telefonul

fiind dispozitivul aflat permanent la îndemână. Din acest motiv, proiectarea interfeței trebuie să țină cont, încă de la început, de constrângerile specifice ecranelor de mici dimensiuni și ale interacțiunii tactile.

Principiul de proiectare adecvat acestui context este cel al designului intuitiv. Interfața se adaptează automat la dimensiunea ecranului pe care este afișată, reorganizând elementele pentru a rămâne lizibile și utilizabile atât pe un monitor de calculator, cât și pe ecranul unui telefon. În cazul unei aplicații de hărți, acest lucru presupune ca harta să ocupe cât mai mult din suprafața disponibilă, iar elementele de control (căutarea, lista de clădiri, detaliile) să fie organizate astfel încât să nu obtureze harta, putând fi afișate sau ascunse după nevoie.

Pe lângă adaptarea vizuală, interacțiunea tactilă impune constrângeri proprii: elementele care pot fi atinse trebuie să fie suficient de mari pentru a fi acționate cu degetul, iar gesturile uzuale, deplasarea hărții prin glisare, mărirea prin apropierea degetelor, trebuie să funcționeze natural. Biblioteca folosită pentru afișarea hărții în ETTIway oferă suport nativ pentru aceste gesturi, ceea ce simplifică obținerea unei experiențe fluide pe dispozitivele mobile.

1.7 Arhitecturi de aplicații web

Platforma ETTIway este construită pe o arhitectură de tip client-server, în care responsabilitățile sunt separate clar între componenta care rulează în browserul utilizatorului și componenta care rulează pe server. Această separare este o practică des întâlnită în dezvoltarea aplicațiilor web moderne.

1.7.1 Separarea frontend-backend

Frontend-ul, adică partea aplicației care rulează în browserul utilizatorului, este responsabil de prezentarea informației și de interacțiunea directă cu utilizatorul: afișarea hărții, gestionarea evenimentelor de tip clic, încărcarea planurilor de etaj și prezentarea detaliilor despre clădiri. Backend-ul, adică partea care rulează pe server, este responsabil de gestionarea datelor, de expunerea serviciilor prin care frontend-ul obține informația și de aplicarea logicii de prelucrare necesare.

Această separare a responsabilităților permite dezvoltarea și testarea independentă a celor două componente. Frontend-ul poate fi modificat fără a afecta logica de date, iar backend-ul poate fi extins cu noi servicii fără a necesita rescrierea interfeței. Comunicarea dintre cele două straturi se realizează printr-o interfață bine definită de servicii, ceea ce reduce cuplarea dintre componente și facilitează evoluția independentă a acestora.

1.7.2 Servicii REST și formatul JSON

Comunicarea dintre frontend și backend în ETTIway se realizează prin servicii de tip REST (*Representational State Transfer*). Acest stil arhitectural se bazează pe protocolul HTTP și organizează accesul la date în jurul noțiunii de resurse. Fiecare resursă este identificată printr-o adresă și manipulată prin operațiile standard ale protocolului. Avantajul acestui model constă în simplitatea pe care o aduce: un serviciu REST poate fi consumat de orice client capabil să efectueze cereri HTTP, indiferent de tehnologia pe care este construit.

Pentru schimbul efectiv de date se folosește formatul JSON (*JavaScript Object Notation*), un format text ușor de citit atât de oameni, cât și de mașini, care permite reprezentarea structurată a obiectelor și a colecțiilor. JSON s-a impus ca format standard în comunicarea dintre aplicațiile web datorită simplității sale și a suportului nativ existent în limbajul JavaScript, folosit pentru dezvoltarea frontend-ului. În cadrul aplicației, datele despre clădiri, etaje și puncte de interes sunt transmise între server și client sub formă de obiecte JSON.

1.7.3 Rețele private virtuale și topologii de tip mesh

Aplicațiile web care lucrează cu date sensibile sau care sunt destinate unor comunități restrânse pot beneficia de o expunere controlată, alternativă expunerii pe internetul public. O astfel de soluție este utilizarea unei rețele private virtuale (VPN, *Virtual Private Network*), care creează un canal criptat între dispozitivele autorizate, izolat de traficul public. În cadrul ETTIway, o asemenea abordare se potrivește atât cu natura limitată a publicului țintă (comunitatea academică a universității), cât și cu cerința de a evita configurări complexe de firewall sau de redirectionare de porturi pe routerul de internet.

Există două topologii principale pentru rețele private virtuale, care diferă prin modul în care traficul circulă între dispozitive. În topologia clasică, de tip **hub-and-spoke** (stea), tot traficul trece printr-un server central, care acționează ca punct de tranzit între toate dispozitivele conectate. Această arhitectură este simplă din punct de vedere conceptual, dar suferă de două

limitări: serverul central este atât un punct unic de eșec, cât și un dezavantaj pentru lățimea de bandă, deoarece tot traficul trebuie să treacă prin el.

Topologia alternativă, de tip **mesh peer-to-peer**, înlocuiește serverul central printr-o rețea în care fiecare dispozitiv poate comunica direct cu oricare altul, fără puncte intermediare obligatorii. În această arhitectură, serverul de coordonare are doar rolul de a anunța dispozitivele unul față de celălalt și de a facilita stabilirea conexiunilor directe, după care nu mai intervine în traficul efectiv. Acest model oferă latențe mai mici, deoarece datele circulă pe drumul cel mai scurt posibil între cele două capete.

Implementarea practică a rețelelor de tip mesh se bazează pe protocoale criptografice moderne, dintre care cel mai răspândit în prezent este **WireGuard**. Acesta este un protocol VPN deschis, conceput cu obiectivul de a oferi performanță ridicată, simplitate de configurare și o suprafață de atac redusă comparativ cu protocoale mai vechi. Cheia identității unui dispozitiv în WireGuard este o pereche de chei criptografice (publică și privată), iar autentificarea se face exclusiv pe baza acestei chei, fără necesitatea unor parole sau certificate centralizate. Deoarece implementarea este compactă (câteva mii de linii de cod, comparativ cu sute de mii pentru soluții mai vechi) facilitează auditul de securitate și reduce probabilitatea unor vulnerabilități.

O provocare importantă pentru rețelele peer-to-peer este **traversarea NAT-urilor** (*Network Address Translation*), procesul prin care dispozitivele aflate în spatele unor routere domestice sau corporative pot stabili conexiuni directe între ele, în pofida faptului că nu au adrese IP publice. Soluțiile moderne folosesc tehnici precum *STUN* (descoperirea adresei externe) și *hole punching* (deschiderea simultană a unor porturi de către ambele capete), iar în cazurile în care traversarea directă nu este posibilă, traficul este redirecționat printr-un server intermediar (relay). Astfel de servere de tranzit sunt folosite ca soluție de rezervă, traficul rămânând criptat capăt-la-capăt, fără ca operatorul relay-ului să poată descifra conținutul.

Această arhitectură de tip mesh, bazată pe WireGuard și pe servicii de coordonare moderne, oferă o alternativă elegantă la expunerea publică a serviciilor. În ETTIway, ea permite ca serverul aplicației să fie accesibil exclusiv dispozitivelor invitate explicit, fără adresă IP publică și fără configurări manuale ale certificatului HTTPS, aspecte detaliate în capitolul de implementare.

1.8 Tehnologii utilizate în implementare

În această secțiune sunt prezentate tehnologiile concrete pe care se sprijină implementarea platformei ETTIway. Alegerea acestor tehnologii a fost ghidată de mai multe criterii: stabilitatea soluțiilor, disponibilitatea unei documentații cuprinzătoare, caracterul open-source al instrumentelor și potrivirea cu cerințele specifice ale unui sistem de navigare în campus.

1.8.1 Spring Boot pentru backend

Backend-ul aplicației este realizat cu Spring Boot, un cadru de lucru (*framework*) pentru limbajul Java, destinat dezvoltării rapide de aplicații și servicii web. Spring Boot reduce semnificativ efortul de configurare necesar pornirii unui proiect, oferind un set de convenții și de componente preconfigurate care permit dezvoltatorului să se concentreze asupra logicii aplicației, nu asupra detaliilor de infrastructură. Printre avantajele sale se numără un server web încorporat, un sistem solid de gestiune a dependențelor și o integrare facilă cu bazele de date.

În cadrul ETTIway, Spring Boot este folosit pentru a expune serviciile REST prin care frontend-ul obține datele despre clădiri și despre infrastructura campusului, precum și pentru a gestiona accesul la stratul de date. Pentru interacțiunea cu baza de date se utilizează mecanisme de persistență de tip JPA (*Java Persistence API*), care permit maparea obiectelor din aplicație

pe înregistrările din baza de date, reducând cantitatea de cod necesar pentru operațiile uzuale de acces la date.

1.8.2 Leaflet și JavaScript pentru frontend

Interfața de vizualizare a hărții este construită folosind biblioteca Leaflet, o bibliotecă JavaScript open-source, dedicată afișării hărților interactive în browser. Leaflet a fost aleasă [5] datorită dimensiunii sale reduse, a performanței bune și a simplității interfeței de programare. Biblioteca oferă suport nativ pentru afișarea hărților bazate pe dale, pentru gestionarea nivelurilor de zoom, pentru adăugarea de markere și pentru organizarea informației pe straturi distincte care pot fi afișate sau ascunse independent.

Restul logicii de pe partea de client este realizat în JavaScript, fără a apela la un cadru de lucru complex pentru interfață. Această alegere a fost motivată de dorința de a păstra aplicația ușoară și de a reduce dependențele externe, în acord cu obiectivul de a obține timpi de răspuns reduși și o experiență fluidă. Interfața este structurată într-un panou lateral, care găzduiește funcțiile de căutare, lista clădirilor disponibile și detaliile clădirii selectate, și o zonă principală ocupată de harta interactivă. Vizualizarea planurilor de etaj se realizează printr-o fereastră modală, care permite parcurgerea nivelurilor unei clădiri fără a părăsi contextul hărții.

1.8.3 Organizarea datelor și extensibilitatea

Datele despre clădiri și despre structura lor interioară sunt organizate într-o formă structurată, care asociază fiecărei clădiri un identificator, denumirea, poziția geografică, o descriere și lista nivelurilor sale, fiecare nivel având asociat un plan corespunzător. Această organizare a fost gândită pentru a fi ușor de extins: adăugarea unei noi clădiri sau a unui nou etaj presupune completarea structurii de date, fără a fi necesare modificări în logica aplicației. Extensibilitatea reprezintă un obiectiv explicit al proiectului, deoarece un campus universitar este un mediu dinamic, în care infrastructura se modifică în timp, iar o soluție viabilă trebuie să poată absorbi aceste schimbări cu efort minim.

1.9 Concluziile capitolului

În acest capitol au fost prezentate fundamentele teoretice și tehnologice care stau la baza platformei ETTIway. Au fost introduse conceptele de sistem de navigare digitală și de hartă interactivă, distincția dintre navigarea exterioară și cea interioară, precum și provocarea integrării acestora într-un flux unitar. S-au discutat apoi conceptele esențiale legate de datele geospațiale, sisteme de referință, reprezentarea geometriilor și sursele de date open-source. A fost analizat mecanismul de poziționare a utilizatorului prin sistemul GPS, împreună cu limitările sale de precizie în mediul construit. Am atins și subiectul algoritmilor de determinare a drumului minim, cu fost prezentate fundamentele algoritmilor de determinare a drumului minim, cu accent pe algoritmul Dijkstra, folosit pentru calculul traseelor. În final, au fost discutate cerințele de proiectare a interfețelor pentru dispozitive mobile și tehnologiile concrete utilizate în implementare. De la cadrul Spring Boot pentru backend până la biblioteca Leaflet pentru vizualizarea hărții, subliniind în fiecare caz motivele care au stat la baza alegerii lor.

Înțelegerea acestor elemente este necesară pentru parcurgerea capitolelor următoare, în care vor fi prezentate analiza cerințelor, arhitectura detaliată a sistemului și modul concret de implementare a funcționalităților platformei. Plecând de la fundamentele expuse aici, următorul capitol abordează cerințele funcționale și nefuncționale ale aplicației, precum și deciziile de proiectare care decurg din acestea.

Capitolul 2

Arhitectura sistemului

Acest capitol prezintă arhitectura platformei ETTIway, plecând de la o vedere de ansamblu asupra componentelor și ajungând la detalierea fiecărui nivel al sistemului. Sunt descrise separarea responsabilităților între frontend și backend. Cat și structura datelor, mecanismul de autentificare și autorizare, sunt prezentate și fluxurile principale prin care componentele comunică între ele. Deciziile de proiectare sunt motivate prin raportare la ideile formulate în capitolul anterior, cu accent pe securitate și claritate a fluxului de utilizare.

2.1 Vedere de ansamblu asupra arhitecturii

Platforma ETTIway este construită pe o arhitectură de tip client-server, organizată pe trei niveluri logice distincte. Primul nivel este cel de prezentare (frontend), realizat în HTML, CSS și JavaScript, care rulează în browserul utilizatorului și se ocupă de afișarea hărții interactive, de interacțiunea cu utilizatorul și de calculul traseelor. Următorul nivel este cel al serviciilor (backend), realizat cu Spring Boot, care expune servicii web, gestionează autentificarea și autorizarea utilizatorilor și mediază accesul la date. Al treilea nivel este cel al datelor, reprezentat de o bază de date relațională PostgreSQL, în care sunt stocate atât conturile utilizatorilor, cât și informația despre structura hărții.

Separarea pe aceste trei niveluri aduce mai multe avantaje. În primul rând, permite dezvoltarea și testarea independentă a fiecărei componente, deoarece interfața dintre niveluri este bine definită. Un alt motiv ar fi ca izolează logica de securitate la nivelul backend-ului, astfel încât regulile de acces nu pot fi ocolite din interfața clientului. În al treilea rând, facilitează extinderea sistemului prin adăugarea de noi servicii sau de noi tipuri de date nu necesită rescrierea componentelor existente, ci doar completarea celor noi.

Comunicarea dintre nivelul de prezentare și cel al serviciilor se realizează prin cereri HTTP către un set de servicii de tip REST, datele fiind transferate în format JSON. Această alegere asigură un cuplaj redus între client și server: orice modificare internă a backend-ului rămâne transparentă pentru frontend, atât timp cât contractul serviciilor este respectat.

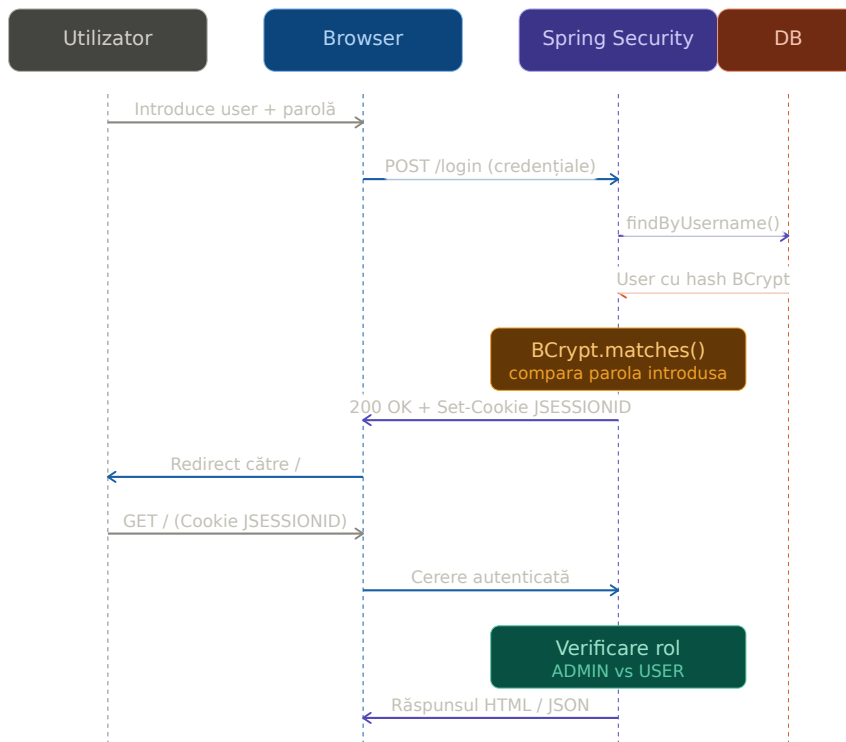


Figura 2.1: Arhitectura sistemului ETTIway pe trei niveluri

2.2 Nivelul de servicii (backend)

Backend-ul aplicației este realizat cu ajutorul framework-ului Spring Boot [4], structurată pe pachete distincte în funcție de responsabilități. Pachetul de configurare conține clasele de inițializare, de securitate și de pornire a aplicației iar pachetul de entități conține clasele care modelează datele. Pachetul de repository conține interfețele de acces la baza de date. Această organizare urmează convențiile de bază ale ecosistemului Spring și face codul ușor de parcurs și de extins.

2.2.1 Punctul de intrare al aplicației

Pornirea aplicației este gestionată de clasa principală, adnotată cu `@SpringBootApplication`. Această adnotare activează mecanismele de configurare automată ale Spring Boot, prin care componentele sunt descoperite și inițializate fără configurare manuală explicită. În ceea ce privește codul punctului de intrare, acesta este minimal, întreaga logică de configurare fiind delegată claselor specializate (vezi Listing A.1 din Anexa A). La pornire, pe lângă inițializarea contextului Spring, aplicația execută și o componentă de inițializare a datelor, care asigură existența unui cont de administrator în baza de date.

2.2.2 Accesul la date prin JPA

Accesul la baza de date se realizează prin intermediul JPA (Java Persistence API) [4]. În locul scrierii manuale a interogărilor, accesul la date este definit prin interfețe care extind `JpaRepository`, Spring generând automat implementările necesare. Pentru utilizator, repository-ul definește în plus o metodă de căutare după numele de utilizator, al cărei corp este generat automat pe baza denumirii metodei (vezi Listing A.2 din Anexa A).

Avantajul acestei abordări este reducerea semnificativă a codului repetitiv: operațiile uzuale de salvare, căutare și ștergere sunt moștenite din `JpaRepository`, iar metodele specifice se declară prin simpla lor semnătură. Tipul de retur `Optional` exprimă explicit faptul că un utilizator căutat ar putea să nu existe, obligând codul apelant să trateze acest caz și reducând astfel riscul de erori.

2.2.3 Strategia de tratare a erorilor

Aplicația trebuie să gestioneze diverse situații de eroare, atât anticipate (date invalide, resurse inexistente, autentificare eșuată), cât și neanticipate (excepții la nivelul bazei de date, erori de rețea). În cadrul ETTIway, tratarea erorilor a fost organizată pe trei niveluri, fiecare cu rol distinct.

La nivelul backend-ului, controllerele REST returnează coduri HTTP standardizate, conform convențiilor REST. Pentru operațiile reușite, codul de răspuns este 200 (OK), pentru resursele inexistente 404 (Not Found), pentru cererile invalide 400 (Bad Request), iar pentru tentativele de acces neautorizat 401 (Unauthorized) sau 403 (Forbidden). Această standardizare face ca frontend-ul să poată trata uniform diferitele tipuri de erori, fără a se baza pe formate proprietare de mesaje.

La nivelul accesului la date, excepțiile aruncate de stratul JPA (de exemplu, violări de constrângeri de unicitate, conexiuni întrerupte) sunt prinse în blocurile try-catch ale serviciilor și transformate în răspunsuri HTTP adecvate. De exemplu, o tentativă de a crea un cont cu un nume de utilizator deja existent este detectată prin excepția de violare a constrângerii `UNIQUE` pe coloana `username` și convertită într-un cod 400.

La nivelul frontend-ului, fiecare apel către API este însoțit de un bloc `.catch` care interceptează erorile de rețea sau erorile returnate de server. În cazul în care o operație eșuează, utilizatorului i se afișează o fereastră modală cu un mesaj clar și acționabil, în loc de a-l lăsa în fața unei interfețe care pare să nu răspundă. Aceste mesaje sunt formulate într-un limbaj accesibil, evitând termenii tehnici care nu ar fi de folos utilizatorului obișnuit.

Această abordare pe trei niveluri urmărește principiul *fail fast* adică erorile sunt detectate și raportate cât mai aproape de sursa lor, fără a permite propagarea unor stări inconsistente prin aplicație. În același timp, prin folosirea codurilor HTTP standard, sistemul rămâne deschis pentru integrarea ulterioară cu alți clienți (de exemplu, o aplicație mobilă nativă).

2.3 Nivelul de date

Datele sunt stocate într-o bază de date relațională PostgreSQL [9], aleasă pentru suportul tipurilor diferite de date și pentru caracterul său open-source. Maparea dintre tabelele bazei de date și obiectele din aplicație este realizată prin adnotări JPA pe clasele de tip entitate. În cadrul ETTIway există două entități principale. Acestea sunt utilizatorii și datele hărții campusului, aceasta din urmă stocând atât informația despre exterior, cât și planurile de etaj din interior.

2.3.1 Alegerea bazei de date

Pe parcursul analizei tehnologice am evaluat mai multe variante pentru componenta de persistență a datelor, comparând atât baze de date relaționale, cât și soluții de tip NoSQL. În acest subcapitol vom sintetiza compromisurile considerate și se va justifica alegerea finală.

Alternativa cea mai evidentă a fost o bază de date relațională clasică, fie PostgreSQL, fie MySQL, ambele oferind maturitate, performanță și un ecosistem solid de unelte și biblioteci. Comparativ, MySQL este mai răspândit în aplicații web mai simple, iar PostgreSQL excelează în suportul pentru tipuri de date avansate, în special pentru câmpurile JSON, pentru indexare flexibilă și pentru extensibilitate. Având în vedere că structura datelor stocate de ETTIway include atât valori scalare clasice (nume utilizator, parolă, rol), cât și documente JSON complexe (graful campusului, planurile indoor), PostgreSQL a fost mai bine adaptat pentru cerințele aplicației.

O a doua opțiune evaluată a fost o bază de date orientată pe documente, cum sunt MongoDB sau CouchDB. Aceste sisteme sunt construite în jurul ideii de stocare flexibilă a documentelor JSON, fără schemă fixă, ceea ce ar fi corespuns natural modelului adoptat pentru graful și planurile indoor. În schimb, ele introduc dependențe suplimentare, complexitatea unei tehnologii separate pentru fiecare tip de date și necesitatea de a gestiona două sisteme diferite în deployment (unul pentru utilizatori și autentificare, unul pentru datele geospațiale). Pentru un proiect de această dimensiune, această diversificare nu se justifică.

Încă o opțiune posibilă, ar fi fost SQLite, fiind o bază de date integrată direct în aplicație, fără server separat, ceea ce simplifică dezvoltarea. Totuși, modelul său de concurență (un singur scriitor activ la un moment dat) o face mai puțin potrivită pentru o aplicație web care trebuie să gestioneze cereri simultane de la mai mulți utilizatori. În plus, SQLite nu oferă mecanismele de gestionare a utilizatorilor și a privilegiilor pe care PostgreSQL le pune la dispoziție nativ.

Astfel, am ales PostgreSQL pentru combinația sa unică: stabilitatea unei baze relaționale tradiționale, suportul pentru documente JSON la nivel de coloană care a permis stocarea flexibilă a grafului, și disponibilitatea ca soluție open-source, gratuită. Mai mult, această alegere lasă deschisă o direcție viitoare de optimizare prin trecerea de la tipul de date `TEXT` (folosit în versiunea actuală) la tipul nativ `JSONB`, care oferă indexare în interiorul documentelor JSON și interogări mai eficiente, fără să necesite o modificare arhitecturală a aplicației.

2.3.2 Entitatea utilizator

Entitatea `User` modelează un cont din sistem și este mapată pe tabelul `users`. Fiecare utilizator are un nume de utilizator unic, o parolă stocată în formă criptată și un rol care determină nivelul de acces. Constrângerile de unicitate și de obligativitate sunt exprimate direct prin adnotări la nivel de coloană (vezi Listing A.3 din Anexa A).

Constructorul fără argumente este necesar pentru ca JPA să poată instanția obiectul atunci când citește o înregistrare din baza de date, iar metodele de tip getter și setter permit citirea și scrierea câmpurilor. Parola nu este niciodată stocată în clar, ci doar în forma sa criptată, aspect detaliat în secțiunea despre securitate.

2.3.3 Entitatea pentru datele hărții

A doua entitate, `MapData`, este responsabilă de persistarea structurii hărții campusului. Pentru a permite o reprezentare flexibilă a grafului de noduri și muchii pe care se bazează rutarea, structura este serializată în format JSON și stocată într-o coloană de tip text. Aceeași entitate cuprinde și planurile de etaj din interior, păstrate într-o a doua coloană JSON distinctă. Această abordare unificată evită multiplicarea tabelelor pentru date conceptual înrudite și permite modificarea structurii grafului sau a planurilor indoor fără a fi necesare migrări ale schemei relaționale (vezi Listing A.4 din Anexa A).

Cele două câmpuri JSON sunt însoțite de un câmp pentru momentul ultimei actualizări, util pentru urmărirea modificărilor aduse hărții. Stocarea flexibilă ca documente JSON constituie o decizie de proiectare orientată spre extensibilitate: noduri, muchii sau planuri de etaj pot fi adăugate ori modificate fără a impune o structură fixă la nivelul tabelelor.

2.3.4 Reprezentarea planurilor de etaj

Planurile de etaj sunt organizate ierarhic în câmpul `indoorJson` al entității `MapData`, sub forma unui obiect care asociază fiecărei clădiri lista etajelor sale. Fiecare etaj conține un document GeoJSON cu toate elementele desenate de administrator: camerele (poligoane închise, cu nume), nodurile (puncte de pe coridoare, având tipuri distincte pentru intrare în cameră, intersecție sau scară) și muchiile (linii care leagă două noduri și reprezintă un segment de coridor). Această organizare păstrează consistența cu modelul folosit pentru harta exterioară și permite reutilizarea aceleiași logici de prelucrare GeoJSON la nivelul clientului.

2.3.5 Strategia de modelare a datelor

Decizia de a stoca atât graful campusului, cât și planurile indoor sub formă de documente JSON serializate ca text, în loc de a folosi o schemă relațională clasică cu mai multe tabele, nu a fost aleasă întâmplător. Modelul ales este similar conceptului de *document store*, utilizat de baze de date precum MongoDB sau CouchDB, dar implementat peste o bază relațională clasică, beneficiind astfel de fiabilitatea PostgreSQL fără a renunța la flexibilitatea schemei.

Avantajul principal al acestei abordări este că structura datelor poate evolua fără modificări de schemă în baza de date. Adăugarea unui nou tip de element (de exemplu, un punct de interes cu proprietăți noi) sau a unui atribut suplimentar pentru o cameră (de exemplu, capacitate, accesibilitate, ore de funcționare) nu necesită alterarea tabelului prin instrucțiuni `ALTER TABLE`. Frontend-ul și logica de prelucrare a JSON-ului sunt singurele componente care trebuie ajustate.

Alternativa relațională ar fi presupus tabele separate pentru clădiri, etaje, camere, noduri și muchii, legate prin chei străine. O astfel de schemă oferă posibilitatea unor interogări complexe la nivelul bazei de date (de exemplu, filtre după proprietăți specifice ale camerelor, agregări per clădire). Acest lucru introduce o rigiditate care nu se justifică în contextul actual al aplicației. Pentru ETTIway, interogările sunt simple: încărcarea în memorie a întregului document și prelucrarea sa la nivelul aplicației. Volumul de date asociat unui campus universitar (câteva sute de clădiri, câteva mii de noduri) face ca această încărcare integrală să fie eficientă, sub un megabyte de JSON pentru un campus de talie medie.

Un al doilea avantaj al modelului este simplitatea persistenței: tot graful exterior sau toate planurile indoor ale unei clădiri sunt salvate printr-o singură operație de scriere în baza de date. Această automatizare evită problemele de consistență care apar într-un model relațional, în care salvarea unui graf complet ar însemna multiple operații de inserare în mai multe tabele, cu necesitatea folosirii tranzacțiilor pentru a garanta integritatea. Aceasta automatizare naturală a stocării JSON face logica de salvare mai simplă atât în backend, cât și în frontend.

2.4 Securitate: autentificare și autorizare

Un aspect central al arhitecturii ETTIway îl constituie mecanismul de securitate, care diferențiază între utilizatorii obișnuiți și administratori. Această diferențiere este esențială, deoarece administratorii pot modifica structura hărții, în timp ce utilizatorii obișnuiți au acces doar în mod de vizualizare. Securitatea este implementată folosind Spring Security [8], integrat nativ în ecosistemul Spring Boot.

2.4.1 Criptarea parolelor

Parolele utilizatorilor nu sunt stocate în clar în baza de date, ci sunt criptate folosind algoritmul BCrypt [10], prin intermediul unei componente dedicate de codare a parolelor. BCrypt este un algoritm de tip funcție de derivare a cheii, conceput special pentru stocarea parolelor: el este lent în mod intenționat și încorporează un factor aleatoriu, ceea ce îngreunează semnificativ atacurile prin forță brută.

1	1	\$2a\$10\$Sj9PXuxG10R.3E0r009UNOLaSoktDIGfKBzS5ehMvFic/9E83put.	ROLE_ADMIN	admin
2	2	\$2a\$10\$IX3vR7Jrl4ZiB85Ybpx/iuu9mUjSvX7XVm8hqYW/xHjpCw6ObSW...	ROLE_ADMIN	student
3	3	\$2a\$10\$GYFuTn4i0ns4woYy9F1.rup7A.QeloQF1jRMWV.woK6v6GQdq.rpi	ROLE_ADMIN	vlad

Figura 2.2: Structura tabelului de utilizatori în baza de date, evidențiind câmpurile pentru numele de utilizator, parola criptată și rolul asociat.

Alegerea BCrypt în detrimentul altor funcții de hash a fost una conștientă. Funcțiile generale de hash criptografic, precum MD5 sau SHA-256, sunt proiectate pentru a fi rapide, ceea ce le face potrivite pentru verificarea integrității fișierelor sau pentru semnături digitale, dar dezavantajoase pentru stocarea parolelor. Viteza lor permite unui atacator cu hardware modern să calculeze miliarde de hash-uri pe secundă, ceea ce face fezabile atacurile prin dicționar pe parole obișnuite. BCrypt, în schimb, este proiectat să fie lent prin construcție și are doi parametri suplimentari care îi conferă rezistența: *salt*-ul și *cost factor*-ul.

Salt-ul este o secvență aleatoare care este atașată parolei înainte de calculul hash-ului. Acest mecanism garantează că două parole identice produc hash-uri diferite și, mai important, blochează atacurile cu *rainbow tables*, care sunt tabele precalculate de hash-uri pentru parolele uzuale. Fiecare hash BCrypt include automat un salt unic, generat aleator la criptare, ceea ce înseamnă că dezvoltatorul nu trebuie să se preocupe explicit de gestionarea sa. Salt-ul este stocat ca parte a hash-ului final, iar la verificare este extras automat pentru a recompune comparația.

Cost factor-ul controlează numărul de iterații pe care le execută algoritmul. La fiecare incrementare a valorii cu 1, calculul durează de două ori mai mult, conform unei progresii exponențiale. Configurația implicită folosită în Spring Security 6 este 10, ceea ce înseamnă $2^{10} = 1024$ de iterații pentru fiecare verificare. Pe hardware modern, această configurație oferă un timp de verificare de aproximativ o sutime de secundă, suficient de rapid pentru un login normal, dar suficient de lent încât atacurile prin forță brută să fie costisitoare. Pe măsură ce hardware-ul devine mai rapid, valoarea acestui parametru poate fi crescută fără modificări structurale ale aplicației sau ale bazei de date.

Componenta de criptare este definită ca un bean Spring (vezi Listing A.5 din Anexa A), ceea ce permite injectarea sa automată în orice altă componentă care are nevoie de criptarea sau verificarea parolelor. La crearea unui cont, parola este trecută prin această componentă înainte de a fi salvată, iar la autentificare, parola introdusă este comparată cu cea criptată din baza de date prin metoda `matches`, care extrage automat salt-ul și cost factor-ul din hash-ul stocat și recompune verificarea.

În cazul în care, la o dată viitoare, BCrypt ar fi considerat depășit, Spring Security oferă mecanisme de migrare progresivă a hash-urilor către un alt algoritm prin componenta `DelegatingPasswordEncoder`. Fără a impune resetarea tuturor parolelor existente. Această flexibilitate, oferită prin prefixarea hash-urilor cu un identificator de algoritm, asigură că soluția adoptată astăzi nu blochează evoluții ulterioare ale platformei.

2.4.2 Configurarea regulilor de acces

Regulile care stabilesc ce resurse sunt accesibile public, ce resurse necesită autentificare și ce resurse sunt rezervate administratorilor sunt definite într-un lanț de filtre de securitate (vezi Listing A.6 din Anexa A). Resursele statice și paginile de autentificare sunt publice, rutele administrative sunt rezervate utilizatorilor cu rol de administrator, iar restul paginilor necesită cel puțin autentificare.

Autentificarea propriu-zisă se realizează printr-un formular de login, configurat să folosească o pagină proprie și să redirecționeze utilizatorul, după succes, către o rută care verifică rolul acestuia și îl direcționează corespunzător. Deconectarea invalidează sesiunea și readuce utilizatorul la pagina de autentificare.

2.4.3 Încărcarea utilizatorilor și inițializarea contului de administrator

Pentru ca Spring Security să poată verifica identitatea unui utilizator, este necesară o componentă care să încarce datele acestuia din baza de date pe baza numelui de utilizator. Această componentă implementează interfața `UserDetailsService` și transformă entitatea proprie `User` într-un obiect pe care mecanismul de securitate îl poate folosi, inclusiv rolul asociat (vezi Listing A.7 din Anexa A).

Pentru a asigura existența unui cont de administrator încă de la prima pornire a aplicației, este folosită o componentă de inițializare care se execută automat la lansare. Aceasta verifică dacă există deja un cont de administrator și, în caz contrar, îl creează, criptând în prealabil parola (vezi Listing A.8 din Anexa A). Astfel, sistemul devine utilizabil imediat după instalare, fără a necesita o configurare manuală a bazei de date. Acest mecanism ilustrează un principiu de proiectare orientat spre ușurința punerii în funcțiune: starea minimă necesară funcționării sistemului este creată automat, reducând efortul de configurare și posibilitatea unor erori manuale.

2.5 Nivelul de prezentare (frontend)

Nivelul de prezentare este realizat în JavaScript, folosind biblioteca Leaflet [7] pentru afișarea hărții interactive. Pe lângă afișarea propriu-zisă, frontend-ul încorporează și o parte importantă din logica aplicației, în special construirea grafului de navigare și calculul traseelor. Această secțiune descrie elementele principale ale interfeței și mecanismele de prelucrare implementate pe partea de client.

2.5.1 Inițializarea hărții și delimitarea zonei

Harta este centrată pe coordonatele campusului și limitată la o zonă geografică bine definită, pentru a împiedica utilizatorul să navigheze în afara regiunii de interes. Coordonatele de centrare și limitele zonei sunt definite ca valori de configurare (vezi Listing B.1 din Anexa B).

Pe lângă afișarea hărții, interfața include un panou lateral care găzduiește funcțiile de căutare, lista clădirilor disponibile și detaliile clădirii. Detaliile despre o sală cum ar fi: clădirea, etajul și tipul sunt generate dinamic, cu prelucrarea atentă a textului pentru a preveni problemele de afișare și eventualele riscuri de securitate, prin transformarea caracterelor speciale înainte de inserarea în pagină.

2.5.2 Construirea grafului de navigare

Pentru a putea efectua calculul traseelor între puncte, infrastructura campusului este transformată într-un graf, în care nodurile reprezintă intersecții sau puncte de pe trasee, iar muchiile reprezintă segmentele care le leagă. Graful este construit pornind de la datele geografice exprimate în format GeoJSON [11], în care elementele de tip linie descriu aleile și drumurile.

Fiecare nod este identificat printr-o cheie obținută din coordonatele sale, rotunjite la o precizie fixă, pentru a asigura că puncte foarte apropiate sunt tratate ca un singur nod. Pentru fiecare segment al unei linii se adaugă o muchie între nodurile adiacente, ponderea muchiei fiind distanța geografică reală dintre cele două puncte. Graful rezultat este nedirecționat, deoarece o alee poate fi parcursă în ambele sensuri. Implementarea construirii grafului este prezentată în Listing B.2 din Anexa B. Observația importantă privind formatul GeoJSON este că acesta exprimă coordonatele în ordinea longitudine–latitudine, în timp ce Leaflet le așteaptă în ordinea latitudine–longitudine. Conversia corectă între cele două este necesară pentru ca punctele să fie plasate corect pe hartă.

2.5.3 Calculul traseului prin algoritmul Dijkstra

Determinarea celui mai scurt traseu între două puncte se realizează prin algoritmul Dijkstra, implementat direct pe partea de client (vezi Listing B.3 din Anexa B). Plecând de la nodul de start, algoritmul menține pentru fiecare nod distanța minimă cunoscută și nodul precedent pe traseul optim. Apoi alege la fiecare pas nodul nevizitat cu distanța cea mai mică și actualizează distanțele vecinilor săi. Atunci când nodul de destinație este atins, traseul este reconstruit parcurgând în sens invers lanțul nodurilor precedente.

Deoarece punctele alese de utilizator nu coincid, în general, exact cu nodurile grafului, este necesară o etapă de asociere a fiecărui punct cu cel mai apropiat nod existent (vezi Listing B.4 din Anexa B). Aceasta se realizează printr-o căutare a nodului aflat la distanța minimă față de punctul țintă, asigurând că rutarea pornește și se încheie în puncte aflate efectiv pe rețeaua de trasee. Dacă între cele două puncte nu există niciun traseu - de exemplu, atunci când o porțiune a rețelei este izolată - algoritmul returnează o valoare care semnalează absența unei rute, iar interfața poate informa utilizatorul în consecință.

2.6 Fluxul de comunicare între componente

Reunind cele trei niveluri, fluxul tipic de utilizare al aplicației poate fi descris astfel. La accesarea aplicației, utilizatorul este direcționat către pagina de autentificare. În cele ce urmează se validează credențialele de către backend, rolul său determină modul de funcționare: vizualizare pentru utilizatorii obișnuiți, respectiv editare pentru administratori. Frontend-ul solicită apoi structura hărții de la backend, care o extrage din baza de date și o transmite în format JSON. Pe baza acestor date, clientul construiește graful de navigare și afișează harta interactivă.

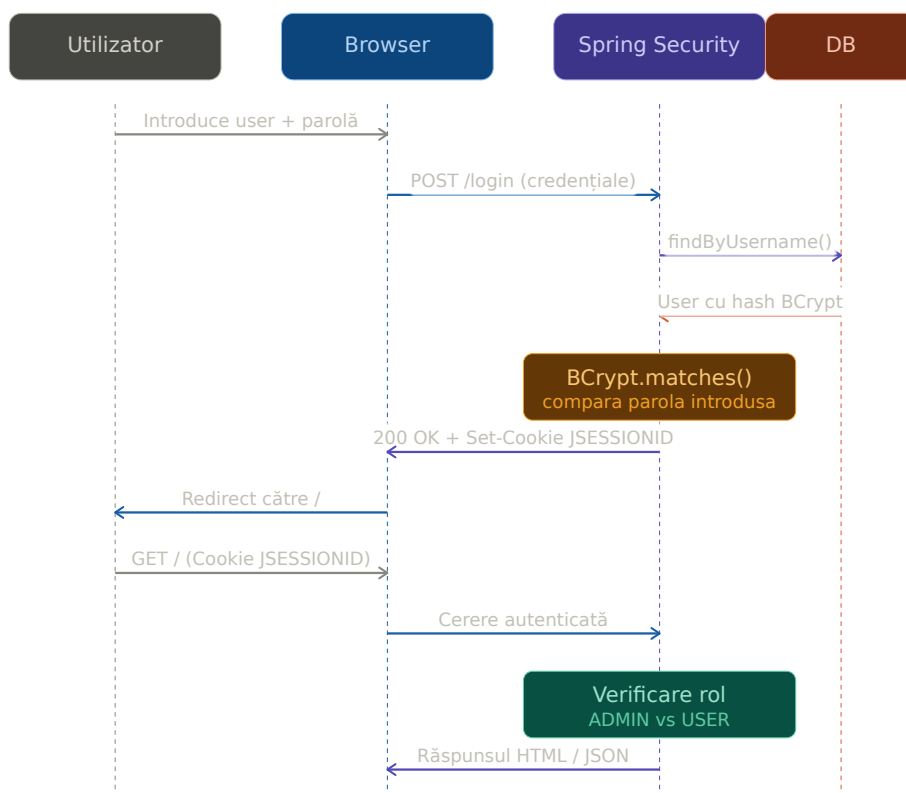


Figura 2.3: Fluxul detaliat de autentificare al unei cereri HTTP

Atunci când utilizatorul selectează un punct de plecare și unul de destinație, calculul traseului se realizează integral pe partea de client, prin construirea grafului, asocierea punctelor cu cele mai apropiate noduri și aplicarea algoritmului Dijkstra. Această alegere reduce sarcina serverului și asigură un timp de răspuns redus. Traseul este calculat și afișat fără a necesita o nouă cerere către backend. Modificările aduse structurii hărții de către un administrator sunt, în schimb, transmise către backend și stocate în baza de date, devenind astfel disponibile tuturor utilizatorilor la accesările ulterioare.

2.7 Concluziile capitolului

În acest capitol a fost prezentată arhitectura platformei ETTIway, organizată pe trei niveluri: prezentare, servicii și date. A fost descris backend-ul realizat cu Spring Boot, cu accesul la date prin JPA și cu stocarea de date în PostgreSQL. Precum și cele două entități principale ale sistemului - pentru utilizatori și pentru datele hărții campusului. Aceasta din urmă cuprinzând atât informația despre exterior, cât și planurile de etaj din interior. S-a detaliat mecanismul de securitate, bazat pe Spring Security, cu criptarea parolelor prin BCrypt. Am prezentat și diferențierea rolurilor și inițializarea automată a contului de administrator. La nivelul de prezentare au fost prezentate inițializarea hărții, construirea grafului de navigare din date GeoJSON și implementarea algoritmului Dijkstra pentru calculul traseelor.

Deciziile de proiectare descrise - separarea pe niveluri, stocarea flexibilă a grafului, calculul traseelor pe partea de client și inițializarea automată a sistemului - au fost orientate spre extensibilitate, securitate și ușurința punerii în funcțiune. Capitolul următor abordează modul concret de utilizare a aplicației și rezultatele obținute, ilustrând funcționarea sistemului în scenariile reale de navigare din campus, incluzând și rutarea în interiorul clădirilor.

Capitolul 3

Implementarea funcționalităților și punerea în funcțiune

După descrierea fundamentelor teoretice și a arhitecturii sistemului, acest capitol arată cum funcționează platforma în practică. Sunt prezentate, pe rând, gestionarea rolurilor de utilizator, editarea hărții de către administrator, căutarea și afișarea informațiilor, măsurile de securitate la nivelul interfeței, rutarea exterioară cu afișarea vizuală a traseului, rutarea în interiorul clădirilor și, la final, infrastructura de rețea care face platforma accesibilă în condiții de siguranță. Fragmentele de cod sunt grupate în Anexa A (backend) și Anexa B (frontend), iar în text se face trimitere la ele acolo unde este cazul.

3.1 Gestionarea rolurilor și modurile de utilizare

Platforma distinge două categorii de utilizatori, fiecare cu un mod propriu de interacțiune. Utilizatorii obișnuiți au acces în mod de vizualizare: pot explora harta, căuta clădiri și săli, consulta informații și calcula trasee. Administratorii au în plus un mod de editare, prin care pot modifica structura hărții campusului. Diferențierea este aplicată la nivelul backend-ului, prin mecanismul de securitate descris în capitolul anterior, astfel încât accesul la funcțiile de editare nu poate fi obținut din interfața clientului fără autentificarea corespunzătoare.

După autentificare, rolul utilizatorului determină cum este încărcată interfața. Pentru un administrator sunt activate controalele de editare, în timp ce pentru un utilizator obișnuit acestea rămân ascunse, interfața fiind orientată exclusiv spre consultare și navigare. Separarea celor două experiențe protejează datele împotriva modificărilor neautorizate și păstrează interfața curată pentru fiecare categorie de utilizatori.

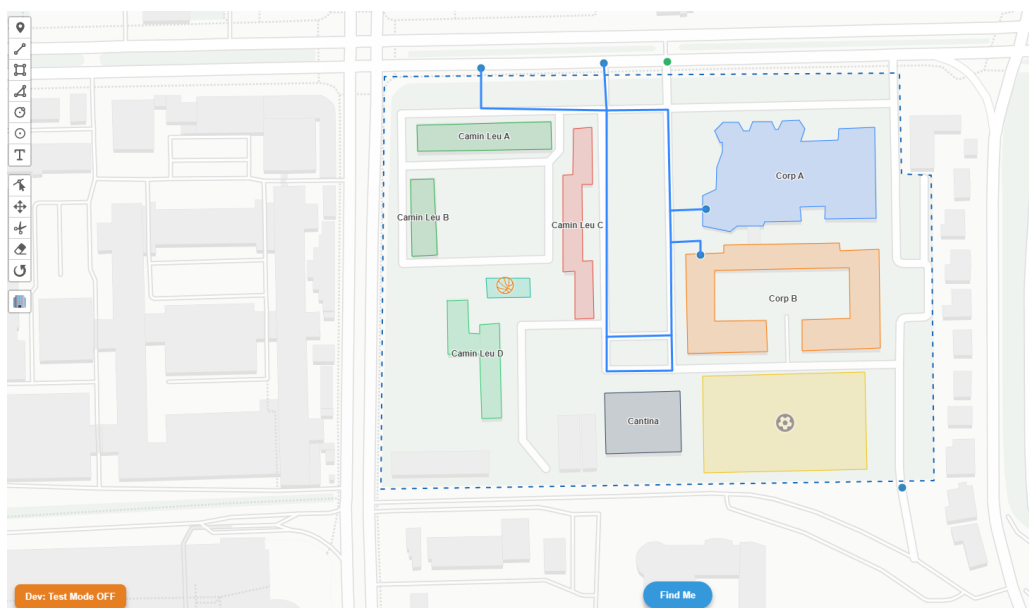


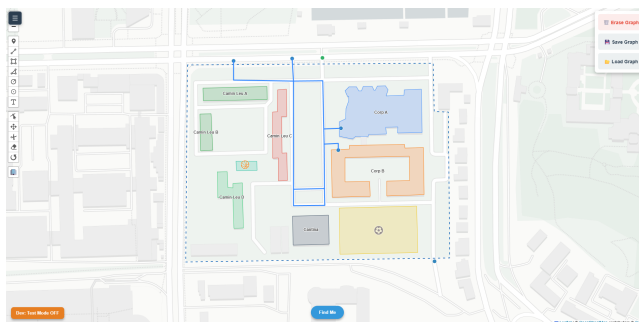
Figura 3.1: Mod vizualizare pagina administratorului

3.2 Editarea hărții de către administrator

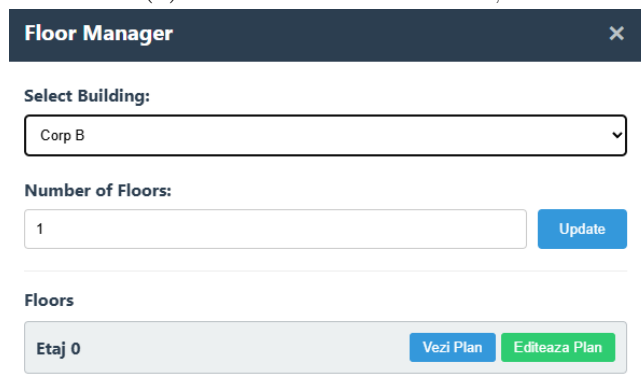
Una dintre funcționalitățile centrale ale platformei este posibilitatea administratorului de a edita structura hărții direct din interfață. În modul de editare, administratorul poate defini și modifica rețeaua de noduri și muchii pe care se bazează rutarea, adăugând puncte de traseu, legături între ele și elemente de infrastructură relevante pentru navigare. Această capacitate este esențială pentru mentenabilitatea sistemului: actualizarea hărții se face vizual, prin interacțiune directă cu harta, fără intervenții în cod sau în baza de date.

Modificările efectuate de administrator sunt transmise către backend prin serviciile dedicate gestionării grafului și sunt persistate în baza de date. Structura hărții, serializată în format JSON, este actualizată în entitatea corespunzătoare și devine imediat disponibilă tuturor utilizatorilor la accesările ulterioare. O modificare făcută de un administrator, adăugarea unui traseu nou sau corectarea unei legături, se reflectă în mod centralizat, fără a fi nevoie de redistribuirea aplicației.

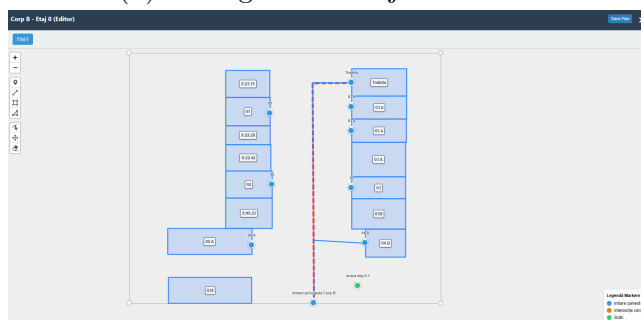
Într-un campus universitar, infrastructura se modifică în timp: apar săli noi, unele se închid pentru lucrări, denumirile clădirilor pot fi actualizate. Capacitatea de a întreține harta rapid, fără efort tehnic, asigură că platforma rămâne utilă pe termen lung.



(a) Modul de editare al hărții



(b) Managerul de etaje interioare



(c) Modul de editare al planului per etaj

Figura 3.2: Fluxul de editare în modul administrator

3.3 Căutarea și afișarea informațiilor

Pentru a permite utilizatorilor să localizeze rapid o sală sau o clădire, platforma include o funcție de căutare integrată în panoul lateral. Căutarea acoperă atât clădirile exterioare, cât și camerele din planurile de etaj indoor, oferind rezultate unificate indiferent de natura locației. Utilizatorul introduce un termen de căutare, iar aplicația identifică elementele corespunzătoare și le afișează, permițând selectarea celui dorit. Pentru utilizatorii nefamiliarizați cu infrastructura campusului, această funcție reduce semnificativ timpul de orientare.

La selectarea unui element, aplicația afișează detaliile asociate. Pentru o sală, sunt prezentate clădirea în care se află, etajul, capacitatea și tipul, însoțite, atunci când există, de o descriere suplimentară. Detaliile sunt generate dinamic și apar fie într-un panou dedicat, fie într-o fereastră de tip popup atașată locației pe hartă, în funcție de contextul interacțiunii. Aceeași logică de generare produce ambele variante, parametrizată după context, ceea ce evită duplicarea codului și asigură consistența informației.

3.4 Securitatea la nivelul interfeței

Pe lângă mecanismele de securitate de la nivelul backend-ului, descrise în capitolul anterior, platforma aplică măsuri de protecție și la nivelul interfeței. O preocupare frecventă în aplicațiile web care afișează date este prevenirea injectării de cod prin conținut nesigur, o categorie de vulnerabilități cunoscută sub numele de cross-site scripting. Problema apare când date provenite dintr-o sursă externă sunt inserate direct în pagină fără prelucrare, ceea ce permite execuția unui cod neintenționat.

Pentru a preveni astfel de situații, valorile text, numele unei săli, descrierea acesteia, sunt trecute, înainte de a fi inserate în pagină, printr-o funcție de prelucrare care transformă caracterele speciale în echivalente inofensive (vezi Listing B.5 din Anexa B). Orice conținut potențial periculos este tratat ca text simplu, nu ca instrucțiuni executabile. Tehnica folosită se bazează pe un mecanism oferit nativ de browser: textul este atribuit unui element printr-o proprietate care îl tratează exclusiv ca text, iar apoi este extras în forma sa deja neutralizată. Soluția este simplă, dar suficientă pentru scenariul de față, și contribuie la robustețea generală a platformei.

O a doua măsură de securitate aplicată la nivelul interfeței privește restricționarea accesului la modul de editare. În versiunile inițiale, activarea controalelor de editare se făcea exclusiv pe baza unui parametru din URL (`?edit=1`), ceea ce însemna că orice utilizator autentificat putea, prin manipularea URL-ului, să intre în interfața de editare. Deși operațiile efective de modificare a hărții erau respinse de Spring Security la nivel de backend, faptul că un utilizator obișnuit vedea totuși controalele de editare crea o impresie incorectă despre permisiunile sale.

Pentru a remedia această situație, am adăugat o verificare suplimentară la nivelul frontend-ului. Înainte de activarea controalelor de editare, codul JavaScript interoghează endpoint-ul `/api/auth/me`, care returnează informațiile despre utilizatorul curent, inclusiv rolul acestuia. Doar dacă rolul returnat este de administrator, interfața este adaptată pentru editare, iar butoanele de salvare devin vizibile. În caz contrar, chiar dacă URL-ul conține parametrul de activare, modul de editare nu este activat și utilizatorului i se afișează un mesaj informativ. Această verificare dublă, pe frontend și pe backend, este un exemplu de aplicare a principiului *defense in depth*, conform căruia protecțiile sunt aplicate pe multiple niveluri, astfel încât eșecul unei verificări nu compromite întreaga securitate.

3.5 Rutarea exterioară și afișarea vizuală a traseului

Rutarea exterioară reunește componentele descrise în capitolul de arhitectură într-o experiență vizuală completă. Utilizatorul selectează un punct de plecare, care poate fi propria poziție obținută prin GPS, și un punct de destinație, iar aplicația calculează și afișează traseul optim între acestea, desenat direct pe hartă.

Procesul se desfășoară integral pe partea de client. Punctele alese sunt mai întâi asociate celor mai apropiate noduri din graful de navigare, după care algoritmul lui Dijkstra determină lanțul de noduri care formează drumul de cost minim. Rezultatul, o secvență de coordonate, este apoi transformat într-o linie trasată pe hartă, suprapusă peste infrastructura campusului, oferind utilizatorului o reprezentare clară a traseului de urmat.

Afișarea vizuală a traseului transformă rezultatul abstract al unui algoritm într-un ghid practic de orientare. Combinată cu poziționarea în timp real a utilizatorului, această funcție permite navigarea efectivă prin campus: pe măsură ce utilizatorul se deplasează, poziția sa este actualizată pe hartă, raportată la traseul calculat. Atunci când nu există un traseu valid, de exemplu între puncte aflate pe porțiuni neconectate ale rețelei, aplicația informează utilizatorul în loc să afișeze un rezultat eronat.

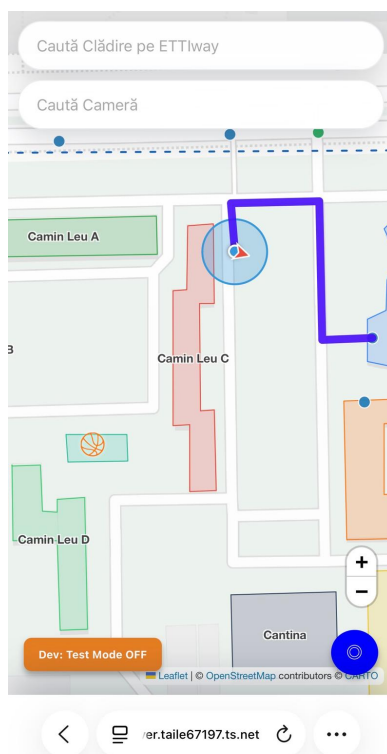


Figura 3.3: Afișarea vizuală a traseului calculat pe hartă cu locația live

3.6 Rutarea în interiorul clădirilor

O componentă importantă a platformei este rutarea în interiorul clădirilor, care extinde experiența de navigare dincolo de granița porților de intrare. Soluția adoptată urmează aceeași abordare ca rutarea exterioară: o rețea de noduri și muchii pe care se aplică algoritmul lui Dijkstra. Diferența ține de modul în care sunt achiziționate datele și de modul în care este reprezentată poziția utilizatorului.

3.6.1 Modelul de date al planurilor de etaj

Fiecare etaj al unei clădiri are propriul plan, stocat ca un document GeoJSON în câmpul `indoorJson` al entității `MapData` descrisă în Capitolul 2. Planul conține trei tipuri de elemente: camere (poligoane închise, cu nume), noduri (puncte de decizie pe coridoare, de tip intrare în cameră, intersecție sau scară) și muchii (linii care leagă două noduri și reprezintă un segment de coridor). Această structură este consistentă cu modelul folosit pentru harta exterioară, doar că lucrează pe un sistem de coordonate plan local, specific fiecărui plan de etaj.

Asocierea dintre o cameră și rețeaua de coridoare se face prin nodul de intrare: lângă fiecare ușă, administratorul plasează un nod de tip „intrare” și îi asociază numele camerei. Astfel, o cameră devine rutabilă: pentru a calcula traseul către ea, este suficient să se cunoască nodul ei de intrare.

3.6.2 Editarea planurilor și salvarea în baza de date

Administratorul construiește planul unui etaj direct din interfață, folosind unelte de desen ale bibliotecii Leaflet.PM. Camerele sunt desenate ca dreptunghiuri sau poligoane, iar la fiecare creare aplicația cere administratorului numele camerei și îl atașează ca proprietate a formei. Nodurile sunt plasate ca markeri, iar tipul lor, intrare, intersecție sau scară, este ales tot printr-un dialog scurt la momentul plasării. Coridoarele sunt desenate ca linii între noduri.

La salvare, întregul conținut al planului este serializat în GeoJSON și trimis către backend printr-o cerere dedicată, care îl persistă în entitatea `indoorJson` al entității `MapData`. La reîncărcare, aplicația cere planul de pe server și îl restaurează vizual împreună cu toate proprietățile. Acest ciclu complet de desenare, salvare și reîncărcare funcționează autonom, fără ca administratorul să fie nevoit să exporte sau să încarce manual fișiere.

3.6.3 Calculul traseului indoor

La nivel de utilizator obișnuit, fluxul de rutare indoor pornește de la căutare. Atunci când utilizatorul caută o cameră, de exemplu „B127”, și o selectează, aplicația identifică automat clădirea și etajul corespunzător și deschide planul respectiv. Aici intervine particularitatea cea mai vizibilă a navigării în interior: GPS-ul nu poate fi folosit pentru a determina poziția utilizatorului, întrucât semnalul satelitar este blocat sau distorsionat de structurile clădirii. Această limitare a fost discutată teoretic în Capitolul 1.

Soluția adoptată este selecția explicită a punctului de plecare. Utilizatorul indică pe planul de etaj unde se află, fie un nod de intersecție pe coridor, fie intrarea unei camere apropiate. Această abordare este folosită inclusiv de aplicații consacrate precum Google Maps în interiorul centrelor comerciale și clădirilor publice, fiind robustă tocmai pentru că nu introduce o precizie artificială pe care tehnologia nu o poate susține. Odată selectat punctul de plecare, aplicația construiește graful etajului din GeoJSON-ul stocat, rulează algoritmul lui Dijkstra între nodul de plecare și nodul de intrare al camerei destinație și desenează traseul rezultat pe plan.

Versiunea actuală tratează rutarea pe un singur etaj. Pentru deplasări între etaje, ar fi necesare noduri de tip scară sau lift, conectate între etaje prin muchii speciale care reprezintă tranziții verticale. Implementarea acestei extinderi este pregătită la nivel de model de date, nodurile de tip scară sunt deja prevăzute, dar conectarea lor între etaje rămâne o dezvoltare ulterioară.

3.6.4 Detectarea dispozitivului și adaptarea interfeței

Având în vedere că utilizarea principală a aplicației se face în deplasare, direct din campus, am acordat o atenție specială experienței pe dispozitive mobile. Spre deosebire de utilizarea de pe laptop sau desktop, navigarea de pe telefon are constrângeri specifice: ecranul mai mic, controlul prin atingere în loc de cursor de mouse, ținerea aparatului cu o singură mână și nevoia ca informațiile critice să fie accesibile rapid.

Pentru a oferi o experiență adaptată, aplicația detectează dispozitivul pe care rulează printr-o verificare simplă a lățimii ferestrei, folosind `window.innerWidth`. Praguri sub valoarea de 768 pixeli sunt considerate dispozitive mobile, ceea ce activează modificări structurale ale interfeței. Această abordare evită nevoia detecției explicite a user-agent-ului, care este fragilă (multe dispozitive moderne raportează agenți modificați), și se bazează pe o proprietate observabilă direct: spațiul vizual disponibil.

Pe dispozitive mobile, panoul lateral cu lista clădirilor și controalele de căutare nu mai este afișat permanent în partea stângă, ci este înlocuit cu un panou de tip *bottom sheet*, care apare dinspre partea de jos a ecranului atunci când utilizatorul interacționează cu el. Acest model este familiar din aplicațiile native mobile (Google Maps, Waze, Uber) și permite ca harta să ocupe cea mai mare parte a ecranului, oferind context vizual maxim, în timp ce informațiile suplimentare sunt accesibile la cerere.

Butoanele și elementele interactive au fost dimensionate respectând recomandarea standard pentru interfețele tactile, cu o suprafață minimă de aproximativ 48 pixeli, suficient de mare pentru a fi atinse confortabil cu degetul fără riscul unor atingeri accidentale pe elementele adiacente. Distanțele dintre elementele acționabile sunt suficiente pentru a evita confuzia în cazul gesturilor rapide.

3.6.5 Gestionarea stării partajate în aplicație

Aplicația frontend este organizată în mai multe module JavaScript independente (harta, căutarea, rutarea exterioară, indoor, autentificarea). Acestea trebuie să poată accesa și actualiza informații comune: graful campusului, datele indoor încărcate, starea de navigare curentă. Pentru a permite această partajare între module, am ales o abordare simplă, fără biblioteci suplimentare, bazată pe obiecte globale expuse pe obiectul `window`. Cum ar fi `window.campusData`, `window.floorData` și `window.navigationData`.

Aplicația este scrisă în JavaScript vanilla, fără framework-uri precum React sau Vue, ceea ce a permis păstrarea unei dependențe minime de cod extern. Toatădată, o foarte bună înțelegere asupra modului în care fiecare componentă se inițializează și interacționează cu celelalte. Pentru un proiect de această dimensiune, modelul cu obiecte globale rămâne lizibil și ușor de urmărit, iar fiecare modul își documentează clar ce câmpuri citește și ce câmpuri actualizează.

Sincronizarea între componente se face prin convenții simple: după ce un modul modifică starea (de exemplu, după salvarea unui plan indoor), apelează direct funcțiile de actualizare ale celorlalte module afectate. Astfel, modificările sunt propagate explicit, fără mecanisme automate, ceea ce păstrează codul predictibil și ușor de depanat.

3.6.6 Verificarea autorizării pe două niveluri

Pe lângă protecția implementată la nivelul backend-ului prin Spring Security (descrișă în Capitolul 2), aplicația realizează o verificare suplimentară a rolului utilizatorului și la nivelul frontend-ului, înainte de a activa modul de editare. Această abordare, cunoscută sub numele de *defense in depth* (apărare pe mai multe niveluri), garantează că un utilizator obișnuit nu

poate accesa interfața de editare nici măcar prin manipularea URL-ului (de exemplu, prin adăugarea parametrului `?edit=1`).

În implementarea actuală, atunci când o pagină este încărcată, codul JavaScript verifică rolul utilizatorului curent printr-un apel către endpoint-ul `/api/auth/me`, care returnează informațiile despre sesiunea activă. Doar dacă rolul returnat este de administrator, controalele de editare sunt adăugate la hartă, iar butoanele de salvare devin vizibile. În caz contrar, chiar dacă URL-ul conține parametrul de activare a editării, interfața rămâne în modul de vizualizare și utilizatorul primește un mesaj informativ care explică restricția.

Această dublă verificare îmbunătățește experiența utilizatorului prin afișarea unor mesaje clare în cazul accesului neautorizat, în loc să afișeze controale care nu funcționează la prima încercare de salvare. În același timp, nu înlocuiește protecția de pe server. Dacă cineva ar manipula codul JavaScript din browser pentru a forța afișarea controalelor, orice încercare de modificare a hărții ar fi respinsă de Spring Security la nivelul endpoint-urilor REST, returnând codul HTTP 403 (Forbidden).

3.7 Punerea în funcțiune prin rețea privată virtuală

Dincolo de implementarea funcționalităților, o componentă importantă a contribuției practice o reprezintă punerea în funcțiune a platformei într-un mediu accesibil de pe dispozitivele mobile ale utilizatorilor. În timpul dezvoltării, întreaga aplicație rulează pe o singură mașină, clientul și serverul comunicând prin interfața de buclă locală (*loopback*). Pentru a deveni utilizabilă în campus, aplicația trebuie să fie accesibilă de pe alte dispozitive, fără a fi expusă riscurilor asociate publicării directe pe internet.

3.8 Configurarea serverului

Serverul gazdă rulează pe distribuția Ubuntu 24.04.4 LTS (*Long Term Support*) [17]. Aceasta a fost aleasă pentru pachetele preinstalate care au ajutat la o integrare mai rapidă a platformei. Printre acestea se numără Java 17.0.19, care ajută la rularea aplicației Spring Boot și PostgreSQL 16.14, care asigură stocarea datelor.

```
=== Sistem ===
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
=== Java ===
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-24.04.2-Ubuntu, mixed mode, sharing)
=== PostgreSQL ===
psql (PostgreSQL) 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)
=== Tailscale ===
1.98.4
```

Figura 3.4: Versiunile principale ale componentelor serverului ETTIway.

Parametrii de rețea include o adresă de IP statică în detrimentul protocolului de alocare dinamic DHCP. Această alegere a fost făcută pentru stabilitatea serverului și pentru a evita probleme de conectivitate la restart sau în cazul în care se întrerupe alimentarea. Gateway-ul implicit este adresa de IP a routerului din rețeaua locală, care asigură conexiunea la internet pentru server.

Pe lângă interfața fizică, serverul rulează clientul Tailscale, care creează o interfață virtuală suplimentară. Aceasta interfață are atribuită o adresă IP `100.X.X.X` rezervată de Tailscale pentru rețele sale private. Această adresă este atribuită din spațiul CGNAT (Carrier-Grade NAT, definit în RFC 6598) care este folosit tocmai pentru a garanta că adresele nu intră

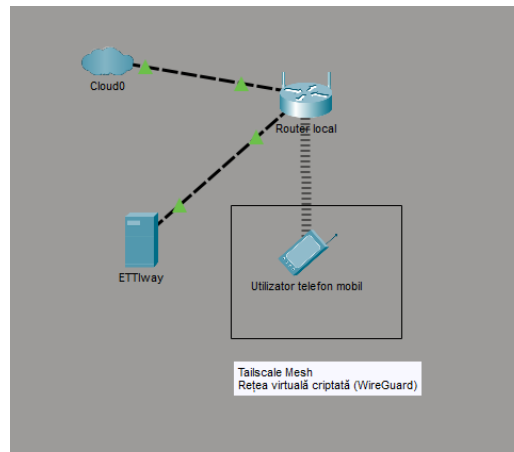


Figura 3.5: Topologia rețelei platformei ETTIway.

în conflict nici cu rețelele publice de internet, nici cu rețelele locale uzuale, eliminând orice ambiguitate de rutare.

Pe serverul gazdă rulează un set restrâns de servicii, fiecare cu un rol bine definit în arhitectura platformei. Tabelul 3.1 prezintă porturile principale pe care ascultă serverul:

Port	Protocol	Serviciu	Acces
22	TCP	SSH (OpenSSH)	Toate interfețele
53	TCP/UDP	DNS (systemd-resolved)	Loopback (cache local)
5432	TCP	PostgreSQL	Doar loopback (127.0.0.1)
8443	TCP	Spring Boot (HTTPS)	Toate interfețele

Tabelul 3.1: Porturile principale ale serverului ETTIway

3.8.1 Rețeaua privată cu Tailscale

Pentru ETTIway, am ales Tailscale [13], o rețea privată virtuală construită peste protocolul de criptare WireGuard [12]. Spre deosebire de un VPN clasic, în care tot traficul trece printr-un server central, Tailscale realizează o rețea de tip *mesh*. Dispozitivele înrolate, serverul pe care rulează aplicația și telefoanele de pe care este accesată, comunică direct între ele, printr-un tunel criptat punct-la-punct (*point-to-point*). Fiecare dispozitiv primește o adresă privată stabilă, validă doar în interiorul rețelei virtuale.

Avantajul principal al acestei abordări este că serverul nu are nicio adresă publică și niciun port deschis către internet. În arhitectura clasică, expunerea unui server presupune redirectionarea porturilor pe ruterul rețelei locale și deschiderea acestora către internet, ceea ce face serverul vizibil și potențial atacabil de oriunde în lume. La ETTIway, această expunere este eliminată. Serverul, împreună cu baza de date PostgreSQL care ascultă pe portul 5432 și cu aplicația Spring Boot, rămâne accesibil exclusiv dispozitivelor autentificate în rețeaua privată. Pentru un proiect administrat de o singură persoană, această configurație reduce semnificativ suprafața de atac.

Pentru a asigura disponibilitatea continuă a aplicației, am configurat-o ca serviciu systemd, ceea ce garantează pornirea automată la repornirea serverului și recuperarea automată în caz de eșec al procesului. Figura 3.6 prezintă starea serviciului, cu indicatorii de uptime, consum de memorie și PID-ul procesului Java.


```
● ettiway.service - ETTIway Application
   Loaded: loaded (/etc/systemd/system/ettiway.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-09 15:51:42 UTC; 1min 59s ago
     Main PID: 1111 (java)
       Tasks: 40 (limit: 19021)
    Memory: 376.6M (peak: 386.1M)
       CPU: 29.660s
    CGroup: /system.slice/ettiway.service
           └─1111 /usr/bin/java -jar /home/vlad18/ETTIway/Backend/demo/target/demo-0.0.1-SNAPSHOT.jar
```

Figura 3.6: Aplicația ETTIway configurată ca serviciu systemd.

3.8.2 Adresarea prin MagicDNS

Pentru a evita memorarea adreselor numerice din rețeaua privată, Tailscale oferă un mecanism propriu de rezolvare a numelor, denumit MagicDNS. Acesta atribuie fiecărui dispozitiv un nume ușor de reținut, sub forma unui subdomeniu în spațiul `.ts.net`. El este valabil doar în interiorul rețelei. Serverul ETTIway este accesat printr-un nume stabil, care rămâne neschimbat indiferent de rețeaua din care se conectează dispozitivele.

3.8.3 Criptarea HTTPS și accesul la GPS

Comunicarea dintre dispozitiv și server este criptată prin protocolul HTTPS, folosind un certificat digital emis automat de Tailscale prin autoritatea Let's Encrypt [14], pentru numele `.ts.net`. Nu este nevoie de un domeniu public sau de o adresă IP publică proprie. Certificatul este recunoscut de browsere ca fiind valid, iar gestionarea sa, emiterea și reînnoirea periodică, este preluată automat de infrastructura Tailscale. Acest lucru este ceea ce reduce la minimum efortul de administrare.

Criptarea HTTPS are aici și o consecință tehnică ce depășește simpla protecție a datelor. Browserele moderne impun ca interfețele sensibile, precum cea de geolocalizare folosită pentru obținerea poziției prin GPS [15], să poată fi utilizate doar într-un context securizat [16], adică numai peste o conexiune criptată HTTPS. Fără o astfel de conexiune, codul JavaScript al aplicației nu ar putea solicita poziția utilizatorului. Ulterior, funcționalitatea de localizare exterioară ar fi indisponibilă. Aici se observă cum aceeași infrastructură care asigură izolarea față de internet oferă, în același timp, certificatul HTTPS necesar pentru accesul la GPS.

3.9 Gestionarea serviciilor pe server

Pe lângă configurarea rețelei și a accesului securizat, un aspect important al punerii în funcțiune este modul în care aplicația este gestionată pe serverul gazdă. Lansarea manuală a aplicației prin comanda `java -jar` și păstrarea ei activă printr-o sesiune SSH deschisă nu este o soluție acceptabilă pentru un serviciu care trebuie să fie permanent disponibil, la prima deconectare, la o repornire accidentală a sistemului sau la o eroare nerecuperabilă, aplicația ar deveni indisponibilă fără intervenție manuală.

Pentru a rezolva această problemă, am configurat aplicația ETTIway cu un manager de servicii standard pe distribuțiile moderne de Linux, serviciul **systemd**. Acesta este primul proces care pornește la inițializarea sistemului de operare și are rolul de a coordona pornirea celorlalte componente, inclusiv a serviciilor definite de utilizator. Configurarea sa se face printr-un fișier text simplu, plasat în directorul `/etc/systemd/system/`, care specifică comanda de pornire, utilizatorul sub care rulează procesul, dependențele de alte servicii și politica de recuperare în caz de eșec.

Pentru ETTIway, fișierul de serviciu definește mai multe aspecte importante. În primul rând, aplicația este configurată să pornească automat după ce sistemul de operare a inițializat rețeaua și serviciul de bază de date, folosind directiva **After**, care exprimă explicit aceste

dependențe. În caz de eșec al procesului Java (de exemplu, o eroare de memorie sau o excepție necaptată), **systemd** repornește automat aplicația după un interval de zece secunde, conform directivei **Restart=on-failure**. Acest comportament transformă o cădere ocazională într-o întrerupere foarte scurtă, fără intervenție manuală.

Personal am ales să rulez aplicația sub un utilizator dedicat (**ettiway**), nu sub utilizatorul **root** care are toate drepturile pe sistem. Această decizie urmărește principiul *least privilege*, adică, chiar dacă ar exista o vulnerabilitate exploatabilă în aplicație, un eventual atacator ar obține doar drepturile limitate ale utilizatorului **ettiway** și nu controlul complet asupra serverului. Această izolare suplimentară reduce semnificativ impactul potențial al unei breșe de securitate.

Un beneficiu suplimentar al configurării ca serviciu **systemd** este integrarea cu jurnalul centralizat **journalctl**. Toate mesajele de output ale aplicației (inclusiv cele generate de Spring Boot la pornire și de Hibernate la fiecare interogare) sunt agregate și pot fi consultate uniform. Comanda **journalctl -u ettiway** afișează istoricul recent al aplicației, iar **journalctl -u ettiway -f** permite urmărirea în timp real a noilor mesaje, util pentru depanare.

3.10 Concluziile capitolului

În acest capitol au fost prezentate implementarea funcționalităților principale și punerea în funcțiune a platformei ETTIway. La nivelul funcționalităților, au fost descrise gestionarea rolurilor de utilizator și administrator, editarea hărții direct din interfață cu persistarea modificărilor în baza de date. Totodată, a fost prezentată și căutarea unificată a clădirilor și a camerelor indoor, măsurile de securitate aplicate la nivelul interfeței pentru prevenirea injectării de cod, rutarea exterioară cu afișarea vizuală a traseului pe hartă și rutarea în interiorul clădirilor, bazată pe planuri de etaj editate de administrator și pe selecția explicită a punctului de plecare.

La nivelul infrastructurii, a fost prezentată abordarea bazată pe o rețea privată virtuală construită cu Tailscale peste protocolul WireGuard, care elimină expunerea serverului pe internetul public și asigură, prin certificatul HTTPS emis automat, contextul securizat necesar accesului la poziția GPS a utilizatorului. Aceste elemente transformă aplicația dintr-un prototip local într-o platformă funcțională, sigură și ușor de administrat, în care securitatea este obținută prin proiectare.

Capitolul 4

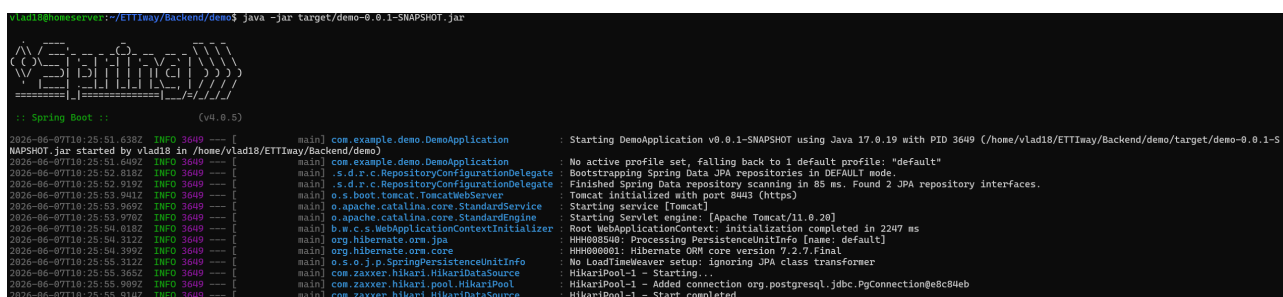
Testarea și evaluarea sistemului

După implementarea funcționalităților și punerea în funcțiune a platformei, descrise în capitolele anterioare, etapa următoare a constat în testarea sistemului pentru a verifica funcționarea corectă a fiecărei componente și comportamentul de ansamblu al aplicației în condiții reale de utilizare. Acest capitol descrie metodologia de testare adoptată, scenariile parcurse, rezultatele observate și limitările identificate. Scopul este de a oferi o evaluare onestă a platformei în starea sa actuală, evidențiind atât aspectele care funcționează conform așteptărilor, cât și pe cele care reprezintă oportunități de îmbunătățire ulterioară.

4.1 Metodologia de testare

Pentru evaluarea platformei ETTIway a fost adoptată o abordare de testare preponderent manuală, orientată spre scenarii reale de utilizare. Această alegere este motivată de natura aplicației: ETTIway este o platformă cu o componentă vizuală și interactivă semnificativă, iar comportamentul său în condiții reale (recepția GPS în campus, fluiditatea interacțiunii cu harta sau claritatea afișării) nu poate fi evaluat automat, ci necesită observare directă pe dispozitivele țintă.

Testarea s-a desfășurat pe două direcții complementare. Prima direcție a vizat verificarea funcțională a fiecărei capabilități în parte. Autentificarea în cele două roluri, căutarea unei săli, afișarea detaliilor, editarea hărții de către administrator, persistarea modificărilor în baza de date și calculul traseului între două puncte. A doua direcție a vizat verificarea de ansamblu a fluxului de utilizare, parcurgând scenariile complete de la deschiderea aplicației până la afișarea unui traseu pe hartă, pentru a observa modul în care componentele conlucrează.



```
vlad@homelab:~/ETTIway/Backend/demo$ java -jar target/demo-0.0.1-SNAPSHOT.jar
:: Spring Boot ::
(v4.0.5)

2026-06-07T10:25:51.638Z INFO 3649 [main] com.example.demo.DemoApplication : Starting DemoApplication v0.0.1-SNAPSHOT using Java 17.0.19 with PID 3649 (/home/vlad18/ETTIway/Backend/demo/target/demo-0.0.1-SNAPSHOT.jar started by vlad18 in /home/vlad18/ETTIway/Backend/demo)
2026-06-07T10:25:51.640Z INFO 3649 [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default profile: "default"
2026-06-07T10:25:52.818Z INFO 3649 [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-06-07T10:25:52.919Z INFO 3649 [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 85 ms. Found 2 JPA repository interfaces.
2026-06-07T10:25:53.941Z INFO 3649 [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8443 (https)
2026-06-07T10:25:53.969Z INFO 3649 [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-07T10:25:53.978Z INFO 3649 [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.20]
2026-06-07T10:25:54.832Z INFO 3649 [main] o.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 2247 ms
2026-06-07T10:25:54.312Z INFO 3649 [main] org.hibernate.orm.jpa : HHH0008540: Processing PersistenceUnitInfo [name: default]
2026-06-07T10:25:54.399Z INFO 3649 [main] org.hibernate.orm.jpa : HHH0008601: Hibernate ORM core version 7.2.7.Final
2026-06-07T10:25:55.312Z INFO 3649 [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-06-07T10:25:55.365Z INFO 3649 [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-06-07T10:25:55.989Z INFO 3649 [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@8c84eb
2026-06-07T10:25:55.914Z INFO 3649 [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
```

Figura 4.1: Pornirea aplicației Spring Boot pe serverul propriu.

Aplicația rulează pe un server propriu, configurat să pornească automat la repornirea sistemului. Figura 4.1 prezintă output-ul tipic al unei porniri reușite, în care se observă inițializarea componentelor Spring Boot.

4.2 Scenarii de testare funcțională

Funcționalitățile centrale ale platformei au fost verificate sistematic, prin parcurgerea unor scenarii reprezentative pentru utilizarea reală. Această secțiune descrie scenariile principale și rezultatele observate.

4.2.1 Autentificarea și diferențierea rolurilor

Pentru verificarea mecanismului de autentificare, s-au parcurs scenarii cu ambele roluri configurate: utilizator obișnuit și administrator. La introducerea credențialelor valide, sistemul redirecționează utilizatorul către interfața corespunzătoare rolului său, încărcând setul de controale adecvat. La introducerea unor credențiale eronate, accesul este refuzat, iar utilizatorul este readus la pagina de autentificare, fără a primi informații care ar putea ajuta la identificarea cauzei refuzului, un aspect important din punct de vedere al securității.

A fost verificată, de asemenea, izolarea funcționalităților rezervate administratorului. Încercarea de a accesa direct, prin manipularea adresei, rutele administrative din contul unui utilizator obișnuit este blocată la nivelul backend-ului, confirmând că diferențierea rolurilor nu se bazează doar pe ascunderea controalelor în interfață, ci este aplicată efectiv la nivelul serviciilor. Această dublă protecție, în interfață și în backend, corespunde practicilor uzuale de securizare a aplicațiilor web.



```
> fetch('/api/graph/save', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({type: "FeatureCollection", features: []})
})
.then(r => {
  console.log("Status:", r.status);
  return r.text();
})
.then(t => console.log("Response:", t));
< Promise {<pending>}
✖ POST http://localhost:8080/api/graph/save 403 (Forbidden)
Status: 403
Response: {"timestamp":"2026-06-09T15:18:46.293Z","status":403,"error":"Forbidden","path":"/api/graph/save"}
```

Figura 4.2: Mesaj afișat în consola la încercarea de salvare a hărții de un cont de user obișnuit

4.2.2 Căutarea și afișarea detaliilor

Funcția de căutare a fost evaluată prin introducerea unor termeni reprezentativi, nume de clădiri și de săli din campus, și observarea rezultatelor afișate. Au fost verificate atât potrivirile exacte, cât și potrivirile parțiale, pentru a confirma că aplicația găsește elementele corecte chiar și atunci când utilizatorul introduce doar o parte a denumirii. La selectarea unui rezultat, detaliile asociate, clădirea, etajul, capacitatea, tipul, sunt afișate complet și consistent, atât în panoul lateral, cât și în ferestrele de tip pop-up atașate locațiilor pe hartă.

4.2.3 Editarea și persistarea modificărilor

Pentru rolul de administrator, a fost verificat ciclul complet de editare a hărții: autentificare ca administrator, intrarea în modul de editare, adăugarea și modificarea unor noduri și legături, salvarea modificărilor și verificarea persistenței acestora. Modificările sunt vizibile imediat după salvare, iar reîncărcarea aplicației, chiar și de pe alt dispozitiv, confirmă că noile date sunt

preluate corect din baza de date și prezentate consistent tuturor utilizatorilor. Acest scenariu confirmă funcționarea fluxului complet între frontend, backend și stratul de persistență.

4.2.4 Calculul și afișarea traseului

Componenta de rutare a fost evaluată prin selectarea unor perechi de puncte distribuite în campus și verificarea traseelor calculate. În toate cazurile în care punctele se află pe porțiuni conectate ale rețelei, algoritmul returnează un traseu, iar acesta este desenat clar pe hartă, urmărind rețeaua de alei și drumuri. Pentru cazurile particulare, punct de plecare identic cu cel de destinație, puncte foarte apropiate, puncte aflate pe componente neconectate ale grafului, comportamentul a fost de asemenea verificat, aplicația tratând corect fiecare situație, fie prin afișarea unui traseu degenerat la un singur punct, fie prin semnalarea absenței unui drum.

4.2.5 Editarea și rutarea în interior

Componenta de rutare indoor a fost verificată folosind date reale: o clădire din campus, pentru care administratorul a desenat planul unui etaj cu mai multe camere, un set de coridoare conectate prin intersecții și nodurile de intrare asociate fiecărei camere. Au fost parcurse, pe rând, cele două roluri ale fluxului indoor.

Pentru rolul de administrator, a fost verificat ciclul complet de editare: desenarea camerelor cu introducerea numelui, plasarea markerilor cu alegerea tipului (intrare cameră, intersecție sau scară), trasarea coridoarelor între markeri și salvarea planului. La reîncărcarea aplicației, planul a fost restaurat fidel din baza de date, incluzând proprietățile fiecărui element și ferestrele de tip tooltip atașate camerelor.

Pentru rolul de utilizator obișnuit, a fost verificat fluxul de căutare și rutare. Căutarea unei camere a deschis automat planul etajului corespunzător, cu camera țintă evidențiată vizual. Selectarea unui punct de plecare din lista de noduri disponibile și apăsarea butonului de calcul al traseului a condus la afișarea drumului optim între punctele alese, urmărind rețeaua de coridoare. Mecanismul de proiectare a nodurilor pe segmente apropiate, descris în Capitolul 3, s-a dovedit util în practică: micile imprecizii din desen, în care o linie nu se conectează exact la un alt segment, sunt tolerate fără a afecta calculul traseului.

Cazul particular al rutării între etaje diferite, neimplementat în versiunea actuală, este tratat explicit de aplicație printr-un mesaj informativ pentru utilizator, evitând rezultate inconsistente.

4.3 Testarea pe mai multe dispozitive și browsere

Deoarece ETTIway este o aplicație web, o cerință importantă este funcționarea corectă pe o varietate de dispozitive și de browsere. Testarea a inclus accesarea aplicației de pe calculatoare personale, prin browserele cele mai răspândite, precum și de pe telefoane mobile cu sisteme de operare diferite. În toate configurațiile testate, aplicația s-a comportat uniform: harta s-a încărcat corect, controalele au răspuns natural la interacțiune, iar funcționalitățile de căutare și rutare au funcționat identic.

Această consistență confirmă alegerea tehnologică de a folosi exclusiv API-uri web standard, fără dependențe de capabilități specifice unui anumit browser. Suportul nativ oferit de biblioteca Leaflet pentru gesturile tactile (deplasarea hărții prin glisare, mărirea prin apropierea degetelor) a contribuit semnificativ la o experiență fluidă pe dispozitivele mobile, care reprezintă contextul principal de utilizare a platformei într-un campus.

4.4 Comportamentul în condiții reale de utilizare

Pe lângă scenariile individuale de testare, platforma a fost folosită în condiții reale, parcurgând efectiv campusul cu un dispozitiv mobil pe care rula aplicația. Acest tip de evaluare, dificil de înlocuit prin teste automate, a oferit informații valoroase despre comportamentul de ansamblu și despre experiența reală a utilizatorului.

În spațiile deschise, în care recepția GPS este bună, poziția utilizatorului afișată pe hartă urmărește fidel deplasarea reală, iar traseele calculate către diferite destinații sunt afișate fluid, în timp real. Tranziția între pașii fluxului de utilizare, autentificare, alegerea destinației, calculul traseului, afișarea sa pe hartă, se desfășoară fără întreruperi vizibile, iar timpul de răspuns al aplicației nu a constituit, în testele efectuate, un factor limitativ.

Trecerea de la spațiul exterior la interiorul unei clădiri, deși nu este tratată automat în versiunea actuală, se face cu o experiență fluidă din perspectiva utilizatorului: o singură interacțiune (deschiderea planului clădirii dorite) marchează tranziția între cele două moduri de navigare, iar contextul vizual al hărții exterioare este păstrat în fundal, oferind un sentiment de continuitate.

În contextul utilizării prin rețeaua privată Tailscale, latența suplimentară introdusă de criptarea VPN și de tranzitul prin serverele de coordonare nu a fost perceptibilă pentru utilizator. Comparativ cu accesarea aplicației direct, prin rețeaua locală, diferențele de timp de răspuns au fost de ordinul zecilor de milisecunde, sub pragul de percepție umană pentru interacțiunile cu o interfață web.

4.5 Limitări identificate

Testarea sistematică și utilizarea în condiții reale au evidențiat și anumite limitări, care merită discutate atât pentru completitudinea evaluării, cât și pentru a identifica direcțiile naturale de dezvoltare ulterioară.

4.5.1 Precizia poziționării în interiorul clădirilor

Limitarea cea mai vizibilă în utilizarea reală este legată de precizia poziționării atunci când utilizatorul se află în interiorul clădirilor. În spațiile deschise, semnalul GPS oferă o precizie de ordinul câtorva metri, suficientă pentru a indica corect poziția pe rețeaua de alei a campusului. În interiorul clădirilor, însă, semnalul satelitar este atenuat și reflectat de structurile de beton și metal, ceea ce conduce la o degradare semnificativă a preciziei, eroarea putând ajunge la zeci de metri.

Această limitare nu este specifică implementării ETTIway, ci este o caracteristică fundamentală a tehnologiei GPS, întâlnită în orice aplicație care depinde de poziționarea satelitară. Ea a fost descrisă, din punct de vedere teoretic, în Capitolul 1, iar testarea practică a confirmat efectul observat și anticipat: funcționarea optimă a localizării este în spațiu deschis, ceea ce corespunde și contextului principal de utilizare a unei aplicații de navigare în campus, adică deplasarea între clădiri, pe alei. Pentru orientarea în interiorul clădirilor, soluții complementare, bazate pe alte tehnologii de localizare, ar putea fi integrate în versiuni viitoare, aspect discutat în capitolul de concluzii.

4.5.2 Acoperirea actuală a datelor

O a doua limitare, distinctă de cea tehnologică, ține de gradul actual de completitudine a datelor introduse în platformă. Aplicația funcționează corect în măsura în care harta și rețeaua de trasee acoperă fidel infrastructura campusului. Extinderea acoperirii, adăugarea de noi clădiri, alei sau puncte de interes, este însă posibilă direct, prin intermediul modului de editare descris în capitolul anterior, fără modificări în cod. Această proprietate face ca limitarea să fie una de scop, nu una structurală, și să poată fi depășită prin muncă incrementală de întreținere a datelor.

4.6 Concluziile capitolului

În acest capitol au fost prezentate metodologia, scenariile și rezultatele testării platformei ET-TIway. Funcționalitățile principale, autentificarea cu diferențierea rolurilor, căutarea, afișarea detaliilor, editarea hărții cu persistarea modificărilor și calculul vizual al traseelor, au fost verificate sistematic, prin scenarii reprezentative, și au fost confirmate ca funcționând conform așteptărilor. Testarea pe mai multe dispozitive și browsere a arătat un comportament uniform, validând alegerile tehnologice care favorizează portabilitatea.

Evaluarea în condiții reale, prin utilizarea aplicației în campus, a confirmat că experiența de navigare este fluidă în spațiile deschise, în care recepția GPS este bună. Singura limitare semnificativă identificată este degradarea preciziei poziționării în interiorul clădirilor, o caracteristică intrinsecă a tehnologiei GPS, care delimitează în mod natural domeniul de utilizare optimă a aplicației la navigarea în spațiile exterioare ale campusului. Această observație, formulată explicit, oferă o bază clară pentru direcțiile de dezvoltare ulterioară, discutate în concluziile finale ale lucrării.

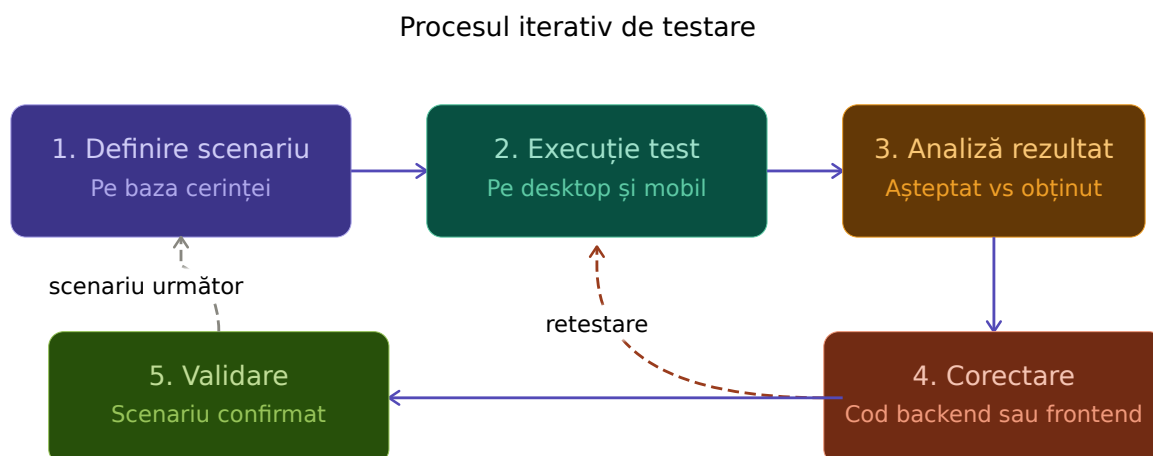


Figura 4.3: Diagrama fluxului de testare a platformei

Concluzii

Lucrarea de față a propus și a realizat ETTIway, o platformă web pentru navigarea interactivă într-un campus universitar, plecând de la o problemă reală, lipsa unei soluții digitale unitare pentru orientarea în campusul ETTI. Totodată, platforma integrează identificarea clădirilor, căutarea sălilor și calculul traseelor între puncte relevante. Această secțiune sintetizează rezultatele obținute, formulează opinia personală asupra acestora și schițează direcții naturale de dezvoltare ulterioară.

Sinteza rezultatelor

Din punct de vedere tehnic, lucrarea a condus la o platformă funcțională, structurată pe trei niveluri logice clar separate: o interfață web construită cu JavaScript și Leaflet, un backend realizat cu Spring Boot și o bază de date PostgreSQL pentru persistarea informațiilor. Au fost implementate funcționalitățile centrale, cum ar fi autentificarea cu diferențierea rolurilor de utilizator și administrator, editarea hărții direct din interfață cu persistarea modificărilor în baza de date și căutarea contextuală a sălilor și clădirilor. Printre care se numără și afișarea detaliilor asociate locațiilor și calculul vizual al traseelor optime între puncte, bazat pe algoritmul lui Dijkstra, combinat cu poziționarea în timp real a utilizatorului prin GPS. Dincolo de navigarea exterioară, platforma include și o componentă de rutare în interiorul clădirilor, în care administratorul poate desena planuri de etaj cu camere, coridoare și noduri de rutare. Ulterior, utilizatorul poate calcula traseul între două puncte de pe același etaj. Soluția folosește același algoritm Dijkstra aplicat unui graf construit din planurile salvate ca documente GeoJSON, ceea ce a permis reutilizarea codului existent fără modificări semnificative.

În planul infrastructurii, soluția adoptată pentru punerea în funcțiune se distinge prin alegerea de a folosi o rețea privată virtuală construită peste WireGuard, în locul expunerii clasice a serverului pe internetul public. Această decizie de proiectare elimină complet suprafața de atac externă a sistemului. Ulterior vom putea expune platforma pentru publicul larg mult mai ușor.

La nivel metodologic, lucrarea a parcurs ciclul complet de dezvoltare a unei aplicații reale: analiza cerințelor, proiectarea arhitecturii, implementarea incrementală, integrarea componentelor, punerea în funcțiune într-un mediu accesibil de pe mai multe dispozitive și testarea sistematică în condiții reale de utilizare. Această experiență, mai mult decât rezultatul tehnic în sine, constituie o componentă importantă a contribuției personale, expunându-mă la deciziile concrete care apar la fiecare nivel al unui sistem software complet.

Opinia personală asupra rezultatelor

Consider că obiectivele declarate ale lucrării au fost atinse în măsură satisfăcătoare. Platforma este funcțională în mediul real, oferă o experiență de navigare fluidă în spațiile exterioare ale campusului și demonstrează că o soluție de orientare digitală poate fi construită folosind tehnologii deschise, fără dependențe comerciale sau costuri de licențiere. Combinarea backend-ului Java cu o interfață web bazată pe JavaScript și Leaflet s-a dovedit potrivită pentru scopul propus, oferind atât performanța necesară, cât și o experiență de utilizator comparabilă cu cea a aplicațiilor de navigare cunoscute.

Cea mai valoroasă lecție a procesului de dezvoltare a fost legată de importanța deciziilor de infrastructură. Inițial, atenția mea s-a concentrat pe funcționalitățile vizibile ale aplicației, harta, căutarea, rutarea, însă în practică s-a dovedit că punerea în funcțiune sigură și corectă a platformei ridică probleme la fel de interesante și solicită soluții la fel de gândite. În particular, înțelegerea legăturii dintre contextul securizat HTTPS și accesul la poziția GPS a fost una dintre observațiile concrete ale proiectului, care a modelat decisiv alegerea soluției de punere în funcțiune.

În același timp, recunosc că platforma, în starea sa actuală, are limite clare. Acoperirea datelor este parțială, iar precizia poziționării în interiorul clădirilor este afectată de constrângerile tehnologice ale GPS-ului, descrise în Capitolul 1 și confirmate experimental în Capitolul 4. Aceste limitări nu sunt eșecuri ale soluției, ci delimitări naturale ale domeniului său actual de aplicabilitate, iar identificarea lor explicită deschide direcții firești de dezvoltare ulterioară.

Direcții viitoare de dezvoltare

Pe baza experienței acumulate și a limitărilor identificate, mai multe direcții de dezvoltare apar ca firești și valoroase.

Prima direcție vizează extinderea funcționalității de navigare indoor către deplasări între etaje diferite. Versiunea actuală tratează rutarea pe un singur etaj la un moment dat, iar selectarea unui punct de plecare și a unei destinații aflate pe etaje diferite este semnalată ca neimplementată. Realizarea acesteia ar presupune adăugarea unor muchii speciale între nodurile de tip scară de pe etaje succesive, precum și a unei logici de tranziție vizuală între planurile etajelor pe măsură ce traseul calculat le traversează. O extindere conexă privește integrarea fluxului exterior–interior: în versiunea actuală, utilizatorul comută explicit între harta campusului și planul unei clădiri, dar un traseu unificat ar putea porni de la o poziție GPS exterioară, ar putea trece prin intrarea principală a clădirii și ar putea continua pe coridoarele indoor până la sala destinație. Pentru poziționarea efectivă a utilizatorului în interiorul clădirilor, unde GPS-ul devine nefiabil, ar putea fi explorate soluții complementare bazate pe balize Bluetooth de joasă energie sau pe semnale Wi-Fi.

A doua direcție privește îmbogățirea acoperirii de date. Adăugarea de noi clădiri, săli, alei și puncte de interes este deja posibilă din interfața de administrare, fără modificări în cod, deci această direcție nu necesită dezvoltare tehnică, ci muncă de întreținere a datelor. Pe termen mediu, ar fi utilă explorarea unor mecanisme de import semiautomat din surse externe, precum OpenStreetMap, pentru a accelera această muncă.

A treia direcție ține de funcționalități avansate de rutare. Algoritmul lui Dijkstra implementat oferă traseul de cost minim, considerând doar distanța, dar ar putea fi extins pentru a ține cont și de alte criterii: accesibilitate pentru persoane cu dizabilități, evitarea zonelor aglomerate, preferința pentru traseele acoperite în caz de vreme nefavorabilă. Asemenea extinderi presupun îmbogățirea grafului cu informații suplimentare la nivelul muchiilor și o interfață prin care utilizatorul să își exprime preferințele.

Una dintre direcțiile care depășește perimetrul strict tehnic este testarea pe scară mai largă, cu utilizatori reali din comunitatea universitară. O astfel de evaluare ar oferi informații pe care testarea individuală nu le poate înlocui, privind ergonomia interfeței, acoperirea reală a nevoilor de navigare și acceptarea soluției ca instrument util în viața de zi cu zi în campus.

Reflecție finală

Dincolo de rezultatul tehnic, lucrarea a reprezentat o experiență în care am avut ocazia să parcurg, în mod integrat, etape pe care, în mod normal, le abordăm separat pe parcursul anilor de studiu: proiectare software, programare web, baze de date, securitate, administrare de sisteme. Această integrare, dificilă la început, mi-a oferit o înțelegere mai matură a modului în care o soluție tehnică reală se construiește prin succesiunea unor decizii la mai multe niveluri, fiecare cu propriile sale considerații și compromisuri.

Pe parcursul proiectului am întâlnit mai multe situații care m-au făcut să înțeleg mai bine echilibrul dintre alegerile teoretice ideale și constrângerile practice. Decizia de a stoca graful campusului sub formă de JSON serializat, și nu într-o schemă relațională cu mai multe tabele, a fost luată după o analiză a complexității necesare versus flexibilitatea câștigată. În cele din urmă, această decizie s-a dovedit corectă, pentru că extinderea către componenta indoor s-a făcut natural, prin adăugarea unei a doua coloane JSON, fără modificări de schemă. La fel, alegerea de a folosi JavaScript vanilla în loc de un framework modern precum React a impus o disciplină mai mare în organizarea codului, dar a păstrat dependențele externe minime și mi-a permis să înțeleg în profunzime fiecare mecanism al aplicației, fără să mă bazez pe capacitățile unui framework.

Una dintre cele mai surprinzătoare lecții a fost legată de aspectele non-funcționale ale proiectului. Inițial, atenția mea s-a concentrat pe funcționalitățile vizibile: cum desenez harta, cum implementez căutarea, cum calculez traseul. Pe parcurs am descoperit că la fel de importante sunt deciziile despre cum punem aplicația în funcțiune, cum o protejăm de accese neautorizate, cum o facem accesibilă de pe dispozitive mobile reale. Descoperirea că accesul la poziția GPS din browser cere obligatoriu un context securizat HTTPS a fost un moment în care am înțeles că proiectarea unei aplicații web moderne presupune o cunoaștere a infrastructurii la fel de solidă ca a algoritmilor de bază. Această observație a modelat decisiv alegerea de a folosi Tailscale, care a rezolvat simultan problema accesului HTTPS automat și pe cea a expunerii sigure a serverului.

Lucrând la indoor, am realizat că soluțiile pentru spațiul interior diferă fundamental de cele pentru exterior, în pofida similarității aparente. Coordonatele pixel ale unui plan desenat manual nu pot fi tratate cu aceleași funcții ca latitudinile și longitudinile reale; toleranța de *snapping* folosită pentru a conecta puncte approximate la rețeaua de coridoare este o soluție pragmatică, iar valoarea de 25 pixeli a fost stabilită experimental, încercând mai multe valori până când rezultatul a devenit predictibil. Astfel de detalii, care nu apar în cărțile despre algoritmi, sunt cele care fac diferența între o demonstrație academică și o aplicație utilizabilă.

Consider că tocmai acest tip de experiență, în care am parcurs toate straturile, de la algoritm la deployment, de la backend la mobile, de la teoretic la practic, este cel mai prețios beneficiu pe care l-am obținut prin elaborarea acestei lucrări. ETTIway nu este doar un produs software, ci și o oportunitate de a verifica cunoștințele acumulate într-un proiect care îmbină multe domenii și care, prin punerea sa în funcțiune în campus, devine o soluție reală, nu doar o exercitare academică.

Bibliografie

- [1] A. Madkour, W. G. Aref, F. U. Rehman, M. A. Rahman, S. Basalamah, *A Survey of Shortest-Path Algorithms*, ResearchGate, 2017.
Disponibil la: <https://www.researchgate.net/publication/316736456>, DOI: 10.48550/arXiv.1705.02044, accesat la data: 1 iunie 2026.
- [2] B. Chen, *Dijkstra's Method with Min-Priority Queue Optimization*, arXiv preprint arXiv:2208.00312, 2022.
Disponibil la: <https://arxiv.org/pdf/2208.00312.pdf>, accesat la data: 1 iunie 2026.
- [3] D. Luxen, C. Vetter, "Real-time routing with OpenStreetMap data", în *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, ACM, 2011, pp. 513-516, DOI: 10.1145/2093973.2094062.
- [4] C. Walls, *Spring in Action*, 6th Edition, Manning Publications, 2022, ISBN 978-1617297571.
- [5] P. Crickard III, *Leaflet.js Essentials*, Packt Publishing, 2014, ISBN 978-1783554812.
- [6] OpenStreetMap Foundation, *OpenStreetMap Wiki: Map Features*,
https://wiki.openstreetmap.org/wiki/Map_Features, accesat la data: 15 mai 2026.
- [7] V. Agafonkin et al., *Leaflet - a JavaScript library for interactive maps*, documentație oficială,
<https://leafletjs.com/reference.html>, accesat la data: 20 mai 2026.
- [8] Spring Team, *Spring Security Reference Documentation*,
<https://docs.spring.io/spring-security/reference/>, accesat la data: 22 mai 2026.
- [9] PostgreSQL Global Development Group, *PostgreSQL Documentation*,
<https://www.postgresql.org/docs/>, accesat la data: 22 mai 2026.
- [10] N. Provos, D. Mazières, "A Future-Adaptable Password Scheme", în *Proceedings of the 1999 USENIX Annual Technical Conference*, USENIX Association, 1999, pp. 81-91.
- [11] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, T. Schaub, *The GeoJSON Format*, RFC 7946, Internet Engineering Task Force, 2016,
<https://datatracker.ietf.org/doc/html/rfc7946>, accesat la data: 18 mai 2026.
- [12] J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel", în *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2017,
<https://www.wireguard.com/papers/wireguard.pdf>, accesat la data: 25 mai 2026.
- [13] Tailscale Inc., *Tailscale Documentation: How Tailscale Works*,
<https://tailscale.com/kb/1151/what-is-tailscale>, accesat la data: 25 mai 2026.

- [14] J. Aas, R. Barnes, B. Case, Z. Durumeric, P. Eckersley, A. Flores-López, A. Halderman, J. Hoffman-Andrews, J. Kasten, E. Rescorla, S. Schoen, B. Warren, “Let’s Encrypt: An Automated Certificate Authority to Encrypt the Entire Web”, în *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2019, pp. 2473-2487.
- [15] Mozilla Developer Network, *Geolocation API*, https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API, accesat la data: 28 mai 2026.
- [16] Mozilla Developer Network, *Secure contexts*, https://developer.mozilla.org/en-US/docs/Web/Security/Secure_Contexts, accesat la data: 28 mai 2026.
- [17] Canonical Ltd., *Ubuntu 24.04 LTS (Noble Numbat) release notes*, <https://documentation.ubuntu.com/release-notes/24.04/>, accesat la data: 11.06.2026.

Anexa A

Cod Java (Backend)

Această anexă reunește fragmentele de cod Java care implementează componentele backend descrise în Capitolele 2 și 3. Numerotarea listing-urilor corespunde trimerilor făcute în text.

Punctul de intrare al aplicației

```
1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class DemoApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(DemoApplication.class, args);
11     }
12 }
```

Listing A.1: Clasa principală a aplicației Spring Boot

Acces la date prin JPA

```
1 package com.example.demo.repository;
2
3 import com.example.demo.entity.User;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import java.util.Optional;
6
7 public interface UserRepository extends JpaRepository<User, Long> {
8     Optional<User> findByUsername(String username);
9 }
```

Listing A.2: Interfata de acces la date pentru utilizatori

Entități

```
1 package com.example.demo.entity;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "users")
7 public class User {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

10     private Long id;
11
12     @Column(unique = true, nullable = false)
13     private String username;
14
15     @Column(nullable = false)
16     private String password;
17
18     private String role;
19
20
21     public User() {}
22
23
24     public Long getId() { return id; }
25     public void setId(Long id) { this.id = id; }
26     public String getUsername() { return username; }
27     public void setUsername(String username) { this.username = username; }
28     public String getPassword() { return password; }
29     public void setPassword(String password) { this.password = password; }
30     public String getRole() { return role; }
31     public void setRole(String role) { this.role = role; }
32 }

```

Listing A.3: Entitatea User mapata pe tabelul users

```

1 package com.example.demo.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.Id;
6 import jakarta.persistence.Table;
7 import java.time.LocalDateTime;
8
9 @Entity
10 @Table(name = "map_data")
11 public class MapData {
12
13     @Id
14     private Long id;
15
16     @Column(columnDefinition = "TEXT")
17     private String graphJson;
18
19     @Column(columnDefinition = "TEXT")
20     private String indoorJson;
21
22     private LocalDateTime updatedAt;
23
24     public MapData() {}
25
26     public Long getId() { return id; }
27     public void setId(Long id) { this.id = id; }
28     public String getGraphJson() { return graphJson; }
29     public void setGraphJson(String graphJson) { this.graphJson = graphJson; }
30     public String getIndoorJson() { return indoorJson; }
31     public void setIndoorJson(String indoorJson) { this.indoorJson =
indoorJson; }
32     public LocalDateTime getUpdatedAt() { return updatedAt; }

```

```

33     public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt =
        updatedAt; }
34 }

```

Listing A.4: Entitatea MapData pentru stocarea hărții exterioare și a planurilor indoor

Securitate: autentificare și autorizare

```

1 @Bean
2 public PasswordEncoder passwordEncoder() {
3     return new BCryptPasswordEncoder();
4 }

```

Listing A.5: Definirea componentei de criptare a parolelor (BCrypt)

```

1 @Bean
2 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
3     http
4         .csrf(csrf -> csrf
5             .ignoringRequestMatchers("/api/graph/**")
6             .disable()
7         )
8         .authorizeHttpRequests(auth -> auth
9
10             .requestMatchers("/login.html", "/auth.html", "/api/auth/**",
11                             "/css/**", "/js/**", "/icons/**", "/data/**").
12             permitAll()
13
14             .requestMatchers("/api/graph/**").permitAll()
15
16             .requestMatchers("/api/admin/**").hasRole("ADMIN")
17
18             .anyRequest().authenticated()
19         )
20         .formLogin(form -> form
21             .loginPage("/login.html")
22             .loginProcessingUrl("/login")
23             .defaultSuccessUrl("/success", true)
24             .permitAll()
25         )
26         .logout(logout -> logout
27             .logoutUrl("/logout")
28             .logoutSuccessUrl("/login.html")
29             .permitAll()
30         );
31
32     return http.build();
33 }
34
35 }

```

Listing A.6: Definirea regulilor de autorizare (extras din SecurityConfig)

```

1 package com.example.demo.config;
2
3 import com.example.demo.entity.User;
4 import com.example.demo.repository.UserRepository;
5 import org.springframework.beans.factory.annotation.Autowired;

```

```

6 import org.springframework.security.core.authority.SimpleGrantedAuthority;
7 import org.springframework.security.core.userdetails.*;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class CustomUserDetailsService implements UserDetailsService {
12
13     @Autowired
14     private UserRepository userRepository;
15
16     @Override
17     public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
18         User user = userRepository.findByUsername(username)
19             .orElseThrow(() -> new UsernameNotFoundException("User not
found"));
20
21         return org.springframework.security.core.userdetails.User
22             .withUsername(user.getUsername())
23             .password(user.getPassword())
24             .authorities(new SimpleGrantedAuthority(user.getRole()))
25             .build();
26     }
27 }

```

Listing A.7: Serviciul de incarcare a utilizatorilor pentru autentificare

```

1 @Configuration
2 public class DataInitializer {
3
4     @Bean
5     CommandLineRunner init(UserRepository userRepository, PasswordEncoder
passwordEncoder) {
6         return args -> {
7             if (userRepository.findByUsername("admin").isEmpty()) {
8                 User admin = new User();
9                 admin.setUsername("admin");
10                admin.setPassword(passwordEncoder.encode(
initialAdminPassword));
11                admin.setRole("ROLE_ADMIN");
12                userRepository.save(admin);
13            }
14        };
15    }
16 }

```

Listing A.8: Initializarea automata a contului de administrator

Persistarea planurilor indoor

```

1 package com.example.demo.repository;
2
3 import com.example.demo.entity.MapData;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface MapDataRepository extends JpaRepository<MapData, Long> {

```


9 }

Listing A.9: Repository-ul pentru entitatea MapData

```
1 package com.example.demo.controller;
2
3 import com.example.demo.entity.MapData;
4 import com.example.demo.repository.MapDataRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.LocalDateTime;
10 import java.util.Optional;
11
12 @RestController
13 @RequestMapping("/api/indoor")
14 public class IndoorMapController {
15
16     @Autowired
17     private MapDataRepository mapDataRepository;
18
19     @PostMapping("/save")
20     public ResponseEntity<String> saveIndoorMap(@RequestBody String
21     indoorJson) {
22         MapData mapData = mapDataRepository.findById(1L).orElse(new MapData
23         ());
24         mapData.setId(1L);
25         mapData.setIndoorJson(indoorJson);
26         mapData.setUpdatedAt(LocalDateTime.now());
27         mapDataRepository.save(mapData);
28         return ResponseEntity.ok("Indoor map saved successfully.");
29     }
30
31     @GetMapping("/load")
32     public ResponseEntity<String> loadIndoorMap() {
33         Optional<MapData> mapDataOpt = mapDataRepository.findById(1L);
34         if (mapDataOpt.isPresent() && mapDataOpt.get().getIndoorJson() !=
35         null) {
36             return ResponseEntity.ok(mapDataOpt.get().getIndoorJson());
37         }
38         return ResponseEntity.ok("{}");
39     }
40 }
```

Listing A.10: Controller REST pentru salvarea si incarcarea planurilor indoor

Anexa B

Cod JavaScript (Frontend)

Această anexă reunește fragmentele de cod JavaScript care implementează componentele frontend descrise în Capitolele 2 și 3. Pentru concizie, unele fragmente sunt prezentate parțial, păstrând doar partea reprezentativă; codul integral este disponibil în repository-ul proiectului.

Configurarea hărții exterioare

```
1 const DEFAULT_CAMPUS_LAT = 44.433215;
2 const DEFAULT_CAMPUS_LON = 26.056764;
3 const DEFAULT_ZOOM_LEVEL = 18;
4 const MAP_BOUNDS = [
5   [44.432102, 26.054500],
6   [44.434329, 26.059028]
7 ];
```

Listing B.1: Parametrii de configurare ai hartii campusului (extras din utils.js)

Construirea grafului și algoritmul Dijkstra (exterior)

```
1 const COORD_PRECISION = 6;
2
3 function getCoordKey(lat, lng) {
4   return `${Number(lat).toFixed(COORD_PRECISION)},${Number(lng).toFixed(
5     COORD_PRECISION)}`;
6 }
7
8 function buildGraphFromGeoJSON(geoJSON) {
9   const graph = {};
10  const nodesMap = {};
11
12  function addNode(lat, lng) {
13    const key = getCoordKey(lat, lng);
14    if (!graph[key]) {
15      graph[key] = [];
16      nodesMap[key] = L.latLng(lat, lng);
17    }
18    return key;
19  }
20
21  function addEdge(key1, key2, dist) {
22    graph[key1].push({ node: key2, weight: dist });
23    graph[key2].push({ node: key1, weight: dist });
24  }
25
26  if (!geoJSON || !geoJSON.features) return { graph, nodesMap };
27
28  geoJSON.features.forEach(feature => {
29    if (feature.geometry.type === 'LineString') {
```

```

29
30     const coords = feature.geometry.coordinates;
31     for (let i = 0; i < coords.length - 1; i++) {
32         const lat1 = coords[i][1], lng1 = coords[i][0];
33         const lat2 = coords[i+1][1], lng2 = coords[i+1][0];
34         const key1 = addNode(lat1, lng1);
35         const key2 = addNode(lat2, lng2);
36         const dist = nodesMap[key1].distanceTo(nodesMap[key2]);
37         addEdge(key1, key2, dist);
38     }
39     } else if (feature.geometry.type === 'Point') {
40
41         const lat = feature.geometry.coordinates[1];
42         const lng = feature.geometry.coordinates[0];
43         addNode(lat, lng);
44     }
45 });
46 return { graph, nodesMap };
47 }

```

Listing B.2: Construirea grafului din date GeoJSON (routing.js)

```

1 function runDijkstra(graph, nodesMap, startKey, endKey) {
2     if (!graph[startKey] || !graph[endKey]) return null;
3
4     const distances = {};
5     const previous = {};
6     const unvisited = new Set();
7
8     for (const node in graph) {
9         distances[node] = Infinity;
10        previous[node] = null;
11        unvisited.add(node);
12    }
13    distances[startKey] = 0;
14
15    while (unvisited.size > 0) {
16        let current = null;
17        let minD = Infinity;
18        for (const node of unvisited) {
19            if (distances[node] < minD) {
20                minD = distances[node];
21                current = node;
22            }
23        }
24        if (current === null || current === endKey) break;
25        unvisited.delete(current);
26
27        for (const neighbor of graph[current]) {
28            if (!unvisited.has(neighbor.node)) continue;
29            const alt = distances[current] + neighbor.weight;
30            if (alt < distances[neighbor.node]) {
31                distances[neighbor.node] = alt;
32                previous[neighbor.node] = current;
33            }
34        }
35    }
36    if (distances[endKey] === Infinity) return null;
37
38    const path = [];
39    let curr = endKey;

```

```

40     while (curr) {
41         path.unshift(nodesMap[curr]);
42         curr = previous[curr];
43     }
44     return path;
45 }

```

Listing B.3: Algoritmul lui Dijkstra (routing.js)

```

1 function findNearestNode(targetLatLng, nodesMap) {
2     let nearestKey = null;
3     let minDistance = Infinity;
4     for (const key in nodesMap) {
5         const dist = targetLatLng.distanceTo(nodesMap[key]);
6         if (dist < minDistance) {
7             minDistance = dist;
8             nearestKey = key;
9         }
10    }
11    return nearestKey;
12 }

```

Listing B.4: Gasirea celui mai apropiat nod din graf (routing.js)

Securitatea la nivelul interfeței

```

1 function escapeHtml(text) {
2     if (text == null) return '';
3     const div = document.createElement('div');
4     div.textContent = String(text);
5     return div.innerHTML;
6 }

```

Listing B.5: Prelucrarea textului înainte de afisare, anti XSS (utils.js)

Indoor: persistare în baza de date

```

1 function saveIndoorDataToDatabase() {
2     fetch('/api/indoor/save', {
3         method: 'POST',
4         headers: { 'Content-Type': 'application/json' },
5         body: JSON.stringify(window.floorData)
6     })
7     .then(response => {
8         if (!response.ok) throw new Error("Failed to save to database");
9         return response.text();
10    })
11    .then(msg => {
12        showCustomAlert("Plan salvat cu succes!");
13    })
14    .catch(error => {
15        console.error("Error saving indoor data:", error);
16        showCustomAlert("Eroare la salvarea planului.");
17    });
18 }

```

Listing B.6: Salvarea planului indoor curent in baza de date (indoor.js)

```

1 function loadIndoorData() {
2     fetch('/api/indoor/load')
3     .then(response => {
4         if (!response.ok) throw new Error("Network response was not ok")
5         ;
6         return response.json();
7     })
8     .then(data => {
9         if (data && Object.keys(data).length > 0) {
10             console.log("Loaded indoor data from database:", data);
11             window.floorData = data;
12         } else {
13             return fetch('data/indoor-data.json')
14                 .then(res => res.ok ? res.json() : {})
15                 .then(localData => {
16                     window.floorData = localData || {};
17                 });
18         }
19     })
20     .catch(error => {
21         console.log("No existing indoor data found or error loading:",
22 error);
23         if (!window.floorData) window.floorData = {};
24     });
25 }

```

Listing B.7: Incarcarea planurilor indoor de pe server la pornirea aplicatiei (indoor.js)

Indoor: conectarea punctelor la rețeaua de coridoare

```

1 function connectNodeToNetwork(nKey) {
2     let p = nodesMap[nKey];
3     let minDist = Infinity;
4     let bestProj = null;
5     let bestEdge = null;
6
7     edgesList.forEach(edge => {
8         if (nKey === edge.k1 || nKey === edge.k2) return;
9         const res = distToSegment(p, edge.p1, edge.p2);
10         if (res.dist < minDist) {
11             minDist = res.dist;
12             bestProj = res.proj;
13             bestEdge = edge;
14         }
15     });
16
17
18     if (bestEdge && (minDist < 25 || nKey === startKey || nKey === destKey))
19     {
20         const projKey = addIndoorNode(bestProj.lat, bestProj.lng);
21
22         const distK1 = euclidean(bestEdge.p1, bestProj);
23         const distK2 = euclidean(bestEdge.p2, bestProj);
24         const distN = euclidean(p, bestProj);
25
26         graph[bestEdge.k1].push({ node: projKey, weight: distK1 });
27     }
28 }

```

```

28     graph[projKey].push({ node: bestEdge.k1, weight: distK1 });
29
30     graph[bestEdge.k2].push({ node: projKey, weight: distK2 });
31     graph[projKey].push({ node: bestEdge.k2, weight: distK2 });
32
33     graph[nKey].push({ node: projKey, weight: distN });
34     graph[projKey].push({ node: nKey, weight: distN });
35 }
36 }

```

Listing B.8: Conectarea nodurilor de start si destinatie la reseaua de coridoare prin proiectie pe segmente (indoor.js)